

# Cost Optimization in Looped Water Distribution Networks

*A Master's Thesis*

Submitted in partial fulfillment of the requirements for the  
degree of Masters of Technology

by

**Shantanu Mapari (22M0796)**

under the guidance of

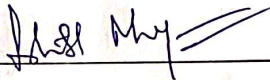
**Prof. Om Damani**



Department of Computer Science and Engineering  
Indian Institute of Technology, Bombay  
Mumbai 400 076

# Dissertation Approval

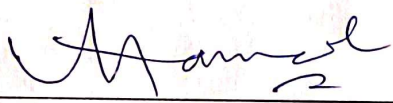
This dissertation entitled **Cost Optimization of Loop Water Distribution Networks** by **Shantanu Mapari** is approved for the degree of **M.Tech.**

Examiners: Prof. Ashutosh Mahajan. 

Prof. Abhiram Ranade 

\_\_\_\_\_

Supervisor: Prof. Om Damani, O. P. Damani

Chairperson: Prof. Abhiram Ranade 

Date: 14 June 2024

Place: IIT Bombay

## DECLARATION

I declare that this written submission represents my ideas in my own words and where others ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Date: 14 June 2024  
Place: IITB

A handwritten signature in black ink, appearing to read 'S. Mapari', with a horizontal line drawn underneath.

(Shantanu Mapari)  
(22M0796)

## Abstract

A water distribution network is essential for delivering water. A water distribution network is essential for delivering water from its source reservoir to numerous endpoints. These networks are fundamental to any society, making it important to seek cost optimization. One practical approach for achieving this optimization is to address the optimal pipe diameter selection problem, which involves determining the most appropriate pipe diameters to be used in various pipe segments of the water network.

JalTantra is a web-based tool that employs both linear programming for branched sections and nonlinear programming for looped sections to optimize water distribution network costs. It achieves this optimization by solving the complex problem of selecting the most efficient pipe diameters throughout the network. The implementation of JalTantra uses various solvers to solve the nonlinear programming problem.

This thesis aims to optimize the network's cost by introducing a flow constraint, i.e., bounding the velocity between nodes by  $v_{max}$ , and analyzing the performance and behavior of various solvers after introducing the constraint. The findings could benefit engineers by offering a more efficient and cost-effective approach to water network design.

Additionally, this work aims to enhance and expand the features of the current JalTantra system. Furthermore, we seek to resolve existing issues, making the JalTantra system more robust and reliable.

# Contents

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>1 Introduction</b>                                       | <b>1</b>  |
| 1.1 Components of water distribution network . . . . .      | 1         |
| 1.2 Types of water distribution network . . . . .           | 2         |
| 1.2.1 Rural water distribution network . . . . .            | 2         |
| 1.2.2 Urban water distribution network . . . . .            | 2         |
| 1.3 Pipe diameter selection problem . . . . .               | 3         |
| 1.4 Report Outline . . . . .                                | 4         |
| <b>2 Background</b>                                         | <b>5</b>  |
| 2.1 Linear and Nonlinear Programming . . . . .              | 5         |
| 2.2 Existing Solvers . . . . .                              | 6         |
| 2.2.1 Knitro . . . . .                                      | 6         |
| 2.2.2 BARON . . . . .                                       | 6         |
| 2.2.3 AlphaECP . . . . .                                    | 6         |
| 2.3 System architecture of Jaltantra Loop version . . . . . | 7         |
| 2.3.1 Frontend architecture . . . . .                       | 7         |
| 2.3.2 Backend architecture . . . . .                        | 8         |
| 2.4 Data input format . . . . .                             | 9         |
| <b>3 Problem Formulation</b>                                | <b>11</b> |
| 3.1 Motivation . . . . .                                    | 11        |
| 3.2 Problem Statement . . . . .                             | 11        |
| <b>4 Methodology</b>                                        | <b>12</b> |
| 4.1 Model details . . . . .                                 | 12        |
| 4.1.1 Model M1 . . . . .                                    | 13        |
| 4.1.2 Model M2 . . . . .                                    | 13        |
| 4.2 Introducing velocity constraints . . . . .              | 15        |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>5</b> | <b>Experimentation</b>                                                | <b>17</b> |
| 5.1      | System Specifications . . . . .                                       | 17        |
| 5.1.1    | Hardware Requirements . . . . .                                       | 17        |
| 5.1.2    | Software Requirements . . . . .                                       | 17        |
| 5.2      | Architecture for running solvers with models . . . . .                | 19        |
| 5.3      | AMPL experimentation setup . . . . .                                  | 19        |
| 5.4      | GAMS experimentation setup . . . . .                                  | 20        |
| <b>6</b> | <b>Improving Velocity Constraint</b>                                  | <b>22</b> |
| 6.1      | Issues with average velocity constraint . . . . .                     | 22        |
| 6.1.1    | Velocity and flow dynamics in pipe segments . . . . .                 | 23        |
| 6.1.2    | Limitations of using average velocity for bounding . . . . .          | 23        |
| 6.2      | Bounding velocity in selected pipe segments . . . . .                 | 24        |
| 6.2.1    | Challenges of bounding velocity in all pipe segments . . . . .        | 24        |
| 6.2.2    | Binary variable to represent the selected pipes in the link . . . . . | 24        |
| 6.2.3    | Determining binary variable constraints . . . . .                     | 25        |
| 6.2.4    | Improving the velocity constraints . . . . .                          | 26        |
| 6.2.5    | Constraints for considerations . . . . .                              | 27        |
| 6.2.6    | Results from testing . . . . .                                        | 27        |
| 6.2.7    | Complexity analysis . . . . .                                         | 28        |
| 6.3      | Optimizing binary variable constraints . . . . .                      | 28        |
| 6.3.1    | The iterative system . . . . .                                        | 29        |
| 6.3.2    | Results of iterative system . . . . .                                 | 32        |
| 6.3.3    | Analysis of why d3 not able to bound the velocity . . . . .           | 34        |
| 6.3.4    | Issues with Iterative System . . . . .                                | 36        |
| 6.4      | Converting binary variable to continuous variable . . . . .           | 37        |
| 6.4.1    | Results after applying continuous variable constraint . . . . .       | 38        |
| 6.5      | Minimum velocity constraints . . . . .                                | 40        |
| 6.5.1    | Testing minimum velocity constraint . . . . .                         | 40        |
| <b>7</b> | <b>Feature Enhancements</b>                                           | <b>43</b> |
| 7.1      | Integrating new velocity constraint in Jaltantra . . . . .            | 43        |
| 7.1.1    | Changes needed to do in code . . . . .                                | 43        |
| 7.2      | Notification improvement . . . . .                                    | 44        |
| 7.2.1    | Key improvements . . . . .                                            | 44        |
| 7.3      | Automation of initial setup of Jaltantra . . . . .                    | 46        |
| 7.3.1    | Challenges . . . . .                                                  | 47        |

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| 7.3.2    | Automation Steps (initial_setup.sh)     | 47        |
| 7.4      | Removing AlphaECP solver from Jaltantra | 48        |
| 7.4.1    | Challenges                              | 48        |
| 7.4.2    | Solution                                | 48        |
| <b>8</b> | <b>Conclusion and Future Work</b>       | <b>50</b> |
| 8.1      | Conclusion                              | 50        |
| 8.2      | Future Work                             | 51        |

# List of Figures

|                                                                                                       |    |
|-------------------------------------------------------------------------------------------------------|----|
| 1.1 Typical Water Distribution System . . . . .                                                       | 2  |
| 1.2 Types of water distribution network . . . . .                                                     | 3  |
| 2.1 Jaltantra Frontend overview . . . . .                                                             | 7  |
| 2.2 Jaltantra Backend overview . . . . .                                                              | 8  |
| 2.3 High-level overview of Jaltantra Constraint checker . . . . .                                     | 10 |
| 5.1 Solver methodology . . . . .                                                                      | 19 |
| 5.2 Architechture for AMPL solvers . . . . .                                                          | 20 |
| 5.3 Architechture for GAMS solvers . . . . .                                                          | 21 |
| 6.1 Link between nodes i & j consist of multiple pipes of varying diameters                           | 22 |
| 6.2 Constraints are applied on violated arcs only . . . . .                                           | 30 |
| 6.3 Flowchart for iterative solution . . . . .                                                        | 31 |
| 6.4 Modified d3 . . . . .                                                                             | 36 |
| 6.5 Knitro Result . . . . .                                                                           | 36 |
| 7.1 Simplified Jaltantra Architecture . . . . .                                                       | 43 |
| 7.2 Current Jaltantra System with issue in notification . . . . .                                     | 45 |
| 7.3 Improved notificaion in case of infeasible solution . . . . .                                     | 45 |
| 7.4 Installation Guidelines for Jalatantra System . . . . .                                           | 46 |
| 7.5 CalculateNetworkCost_JaltantraLauncher.sh (Script to invoke solver<br>and run analysis) . . . . . | 48 |

# List of Tables

|                                                                                                                 |    |
|-----------------------------------------------------------------------------------------------------------------|----|
| 5.1 Dataset information . . . . .                                                                               | 18 |
| 6.1 Results of velocity constraints testing . . . . .                                                           | 28 |
| 6.2 Effect of applying velocity constraints on number of variables and<br>total number of constraints . . . . . | 29 |
| 6.3 Maximum and minimum velocities in each network file without ve-<br>locity constraints . . . . .             | 33 |
| 6.4 Bound velocity with $v_{max}$ 20m/s . . . . .                                                               | 33 |
| 6.5 Bounding velocity with $v_{max}$ 15m/s . . . . .                                                            | 34 |
| 6.6 Bounding velocity with $v_{max}$ 10m/s . . . . .                                                            | 35 |
| 6.7 d3 Analysis to find minimum diameter required . . . . .                                                     | 35 |
| 6.8 Continuous variable constraint result . . . . .                                                             | 39 |
| 6.9 Continuous variable velocity constraint results on Indian WDN . .                                           | 39 |
| 6.10 Minimum velocity in all the data files . . . . .                                                           | 41 |
| 6.11 Result of applying minimum velocity constraints . . . . .                                                  | 41 |

# Chapter 1

## Introduction

Water distribution systems are essential to modern infrastructure that delivers clean and safe drinking water to communities worldwide. These systems consist of a network of pipes, valves, pumps, and storage tanks that transport water from treatment plants to homes, businesses, and other facilities. The main objective of the Water Distribution System is to provide the necessary amount of water to demand nodes satisfying cost and pressure constraints.

Various optimizations are done in water distribution, such as Cost Optimizations, Schedule Optimizations, Resilience Optimizations, and District metered zone formulation optimizations. Our primary focus is on Cost optimization by pipe diameter selection problem. We also aim to introduce new constraints for bounding the velocity between the nodes provided by the design engineer and analyze the results of various non linear programming solvers. Previous work has been done to find out suitable solvers to get the optimal pipe diameter for designing the water distribution networks.

### 1.1 Components of water distribution network

A standard water distribution network consists of mass-balancing reservoirs, demand nodes, and pipes connecting the nodes. Tanks are introduced to act as a buffer between primary source and demand nodes. Tanks divide the network into two parts, primary and secondary. Rural water networks are mostly gravity-fed, and pumps are installed at the source to provide the necessary pressure. Flow regulating devices such as valves are installed at the pipeline to manage the flow rate. The network in consideration is a branch acyclic single source network. A typical water distribution network is shown in the figure [1.1](#)

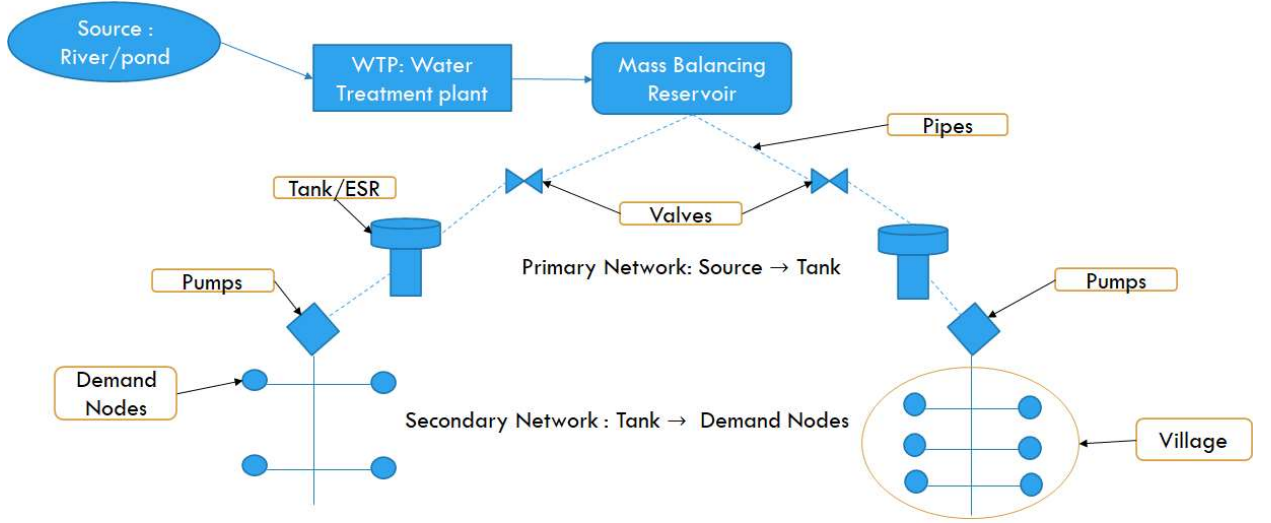


Figure 1.1: Typical Water Distribution System

## 1.2 Types of water distribution network

There are two types of water distribution networks namely rural water distribution networks and Urban water distribution networks.

### 1.2.1 Rural water distribution network

Rural water distribution networks are typically gravity-fed, single-source systems with a branching, acyclic structure. This linear structure ensures that the objective function and constraints are linear, allowing for applying linear programming techniques. This, in turn, provides deterministic answers and the ability to find global minima when optimizing the network's design and operation. This linear approach enhances the efficiency and reliability of water supply in remote areas. The Jal-tantra branch version is dedicated to optimizing the cost of designing a rural water distribution network. Fig 1.2a shows typical rural water distribution network

### 1.2.2 Urban water distribution network

Urban water distribution networks are characterized by looped structures, which introduce complexity due to non-linear flow constraints. This complexity poses a

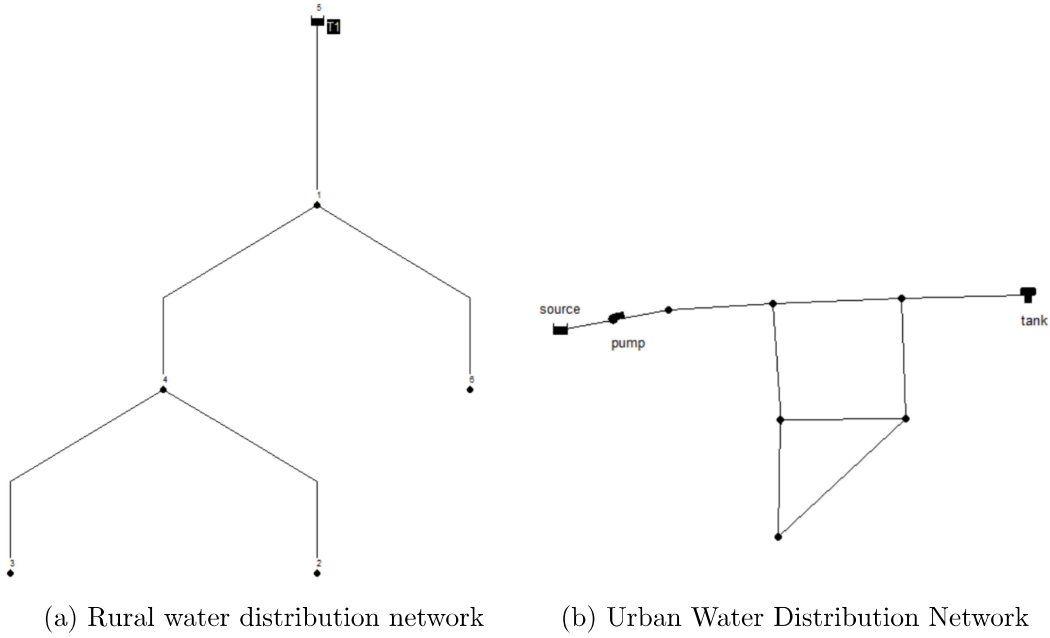


Figure 1.2: Types of water distribution network

significant challenge in finding solutions, requiring specialized solvers to navigate the network's intricacies. The Jaltantra Loop version is used to optimize urban water distribution networks, facilitating the search for local minima and enhancing the efficiency and reliability of water supply in urban areas. Fig [1.2b](#) shows an urban water distribution network.

### 1.3 Pipe diameter selection problem

Water distribution network costs can be reduced by carefully choosing the optimal pipe diameters. Larger pipes minimize head loss (energy loss due to friction) but come with higher installation costs. Conversely, smaller pipes are more affordable but result in more significant head loss. Therefore, the challenge lies in selecting the proper pipe diameters for each network segment, balancing cost considerations with the need to meet constraints such as maintaining minimum pressure, satisfying demand, and ensuring minimum flow rates for consumers.

## 1.4 Report Outline

Further chapters include Background. In this chapter, we will describe linear and nonlinear programming, the existing solvers, and the system architecture of Jaltantra. Also, we will explain the format of the datasets we will use. In the next chapter, we will describe the problem statement. In this section, we will have an overview of the cost optimization problem and motivation. In further chapters, we will have a detailed overview of the methodology for introducing the velocity constraint with various models. In the Experimentation chapter, we will see how we will perform the experiments on various parameters. Further, we will improve the velocity constraint and prove its correctness through results and analysis. In the next chapter, we will review some feature enhancements/bug fixing and how to integrate the new constraint in the currently deployed system. In the last chapter, we will discuss the conclusion and future work.

# Chapter 2

## Background

This chapter will overview the Jaltantra Loop version, which is used to optimize cost for looped water distribution networks, and how linear and nonlinear programming techniques are used. Then, we will understand the system architecture of the currently deployed Jaltantra system.

### 2.1 Linear and Nonlinear Programming

- **Linear Programming :** Linear programming is a deterministic method that ensures optimal solutions for problems involving continuous variables and linear objective functions, given linear constraints. However, we encounter nonlinear relationships in water distribution systems, specifically between flow rate and head loss (water loss due to pipe friction), which is inherently quadratic. Consequently, if we wish to employ linear programming techniques, we need to linearize these nonlinear aspects.
- **Nonlinear Programming :** Nonlinear programming extends the optimization framework to problems where either the objective function, constraints, or both are nonlinear. These methodologies are generally more challenging than linear programming because they can exhibit multiple locally optimal solutions, making it difficult to ensure the existence of a globally optimal solution.

## 2.2 Existing Solvers

As Nonlinear Programming problems are complex, we need external solvers to get the best local solution. In the previous work done in Jaltantra [7], three solvers, namely Knitro, Alphaecp, and Baron, looked promising and have been integrated into the Jaltantra looped version. Still, we did not own a commercial license. We must thoroughly analyze these solvers to decide which solver to go with.

### 2.2.1 Knitro

Knitro is a commercial optimization software package developed by Artelys that is widely used to solve nonlinear programming (NLP) problems. In Jaltantra-loop, Knitro is used with **AMPL** based model files.

The Knitro solver provides a range of algorithms to address different problem types. For continuous problems, it offers options like direct interior point, conjugate gradient interior point, and sequential linear-quadratic active-set algorithms. In the case of mixed-integer problems, Knitro provides both general and specialized convex branch-and-bound procedures, utilizing any of the three continuous algorithms to solve subproblems. It uses a concurrent multistart method to discover high-quality local optima for nonconvex problems.

### 2.2.2 BARON

BARON (Branch-and-Reduce Optimization Navigator) is also a commercial optimization solver. One of BARON's primary strengths is its global optimization capabilities. It employs advanced global optimization algorithms to search for the global optimum in nonconvex and mixed-integer problems.

BARON uses a branch-and-bound algorithm as its core optimization method. This technique systematically explores the solution space by partitioning it into smaller regions and bounding the objective function within those regions. It iteratively refines the bounds and searches for the global optimum. BARON also uses AMPL-based models and data files.

### 2.2.3 AlphaECP

AlphaECP is a commercial Mixed-Integer Nonlinear Programming (MINLP) solver based on the GAMS modeling language. AlphaECP employs a range of advanced optimization algorithms to tackle MINLP problems. These algorithms include

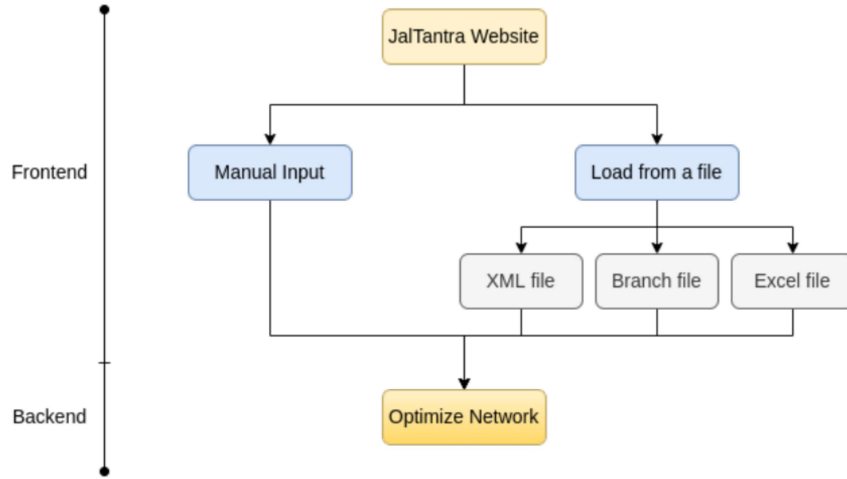


Figure 2.1: Jaltantra Frontend overview

branch-and-bound, branch-and-cut, and branch-and-price techniques. These methods help explore the solution space efficiently and find high-quality solutions.

While AlphaECP is particularly effective for global optimization, it can also handle local optimization problems. Although it may be computationally intensive, global optimization is crucial for finding the global optimum in nonconvex situations.

## 2.3 System architecture of Jaltantra Loop version

This section will give an in-depth overview of the currently deployed Jaltantra loop version. It consists of components such as frontend overview, backend overview, and constraint analyzer.

### 2.3.1 Frontend architecture

Figure [2.1](#) gives a high-level overview of the JalTantra website's front end. The end user's primary task involves inputting data related to their pipeline network. This data can be entered manually or loaded from pre-existing files, such as XML files, Excel files, or Branch files. Once the network data is successfully entered or loaded onto the website, all that's left for the user to do is click the "Optimize"

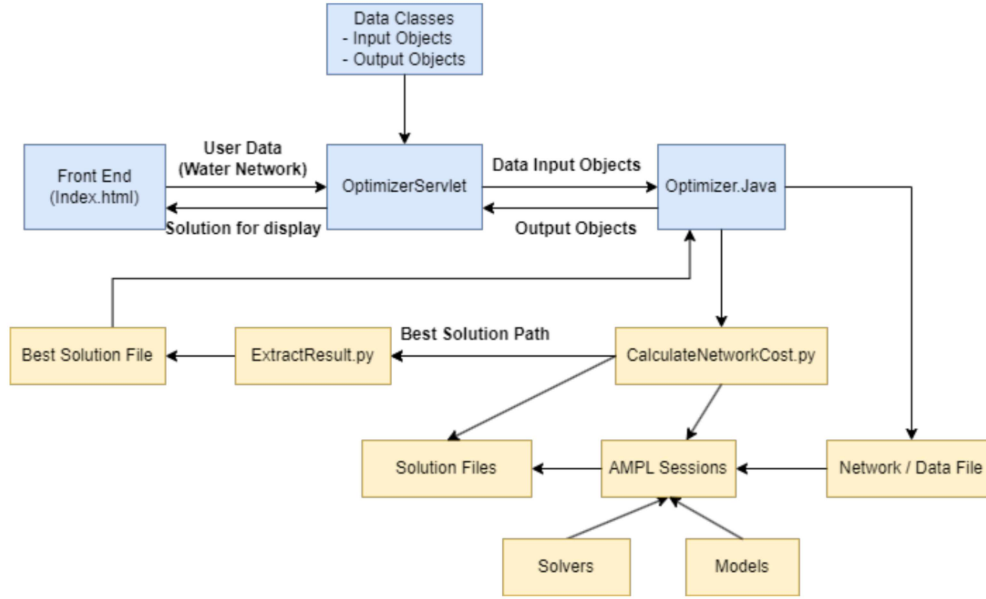


Figure 2.2: Jaltantra Backend overview

button. This action sends a request to the backend, prompting it to initiate the optimization process for the network.

### 2.3.2 Backend architecture

Referring to Figure 2.2, here's the step-by-step process that happens when a user clicks the "Optimize" button in the Jaltantra system:

- **Data Conversion and Transfer :** When the user hits "Optimize," the frontend takes their data and turns it into JSON (JavaScript Object Notation). This JSON data is then sent to the backend, where the "doPost(...)" method in the Optimizer servlet receives it.
- **Optimizer Servlet :** The Optimizer servlet converts the data into relevant data objects (Java Objects), including Pipes, Nodes, and Commercial Pipes, each with specific properties, and sends this data to Optimizer.java for further processing.
- **Optimizer.java :** The system first checks if the user's submitted network is valid by running some reviews. If everything looks good, a network file

is created with SHA-256 hash on the server, and this file is crucial for the optimization process with solvers. Using the SHA-256 hash of the network file, the system checks whether this network has been submitted and optimized before. If it's a new network (no status file exists), the system asynchronously launches "CalculateNetworkCost\_JaltantraLauncher.sh." This script, in turn, gives the network file to "CalculateNetworkCost.py," launching solvers to find the best solution.

- **CalculateNetworkCost.py** : This program is responsible for the heavy lifting, as it launches multiple solvers to work on various model files in parallel. The program starts solvers in two batches using Tmux. It checks for errors, gives extra time, and manages the transition to the second batch. After everything is done, it saves the best solution, updates the status file, and finishes.
- **ExtractResult.py** : This program uses the solution path given by "CalculateNetworkCost.py" to create the best solution file in a compatible format for Optimizer.java. Optimizer.java processes this output and translates it into output objects. These output objects are then sent to the OptimizerServlet as an intermediary. Ultimately, the results are presented on the front end for the user.

### High-level overview of Constraint checker

Before making the solution file for "Optimizer.java," the solution is carefully checked. At each step, the code looks to see if the solution follows our set constraints. If it breaks any of these constraints, the code marks it as invalid with an exit code of -1, so it won't be used, as shown in fig [2.3](#).

## 2.4 Data input format

The input format given in the Jaltantra system can be manually or given in Excel, branch, or XML files. This input is converted to a network file. In Jaltantra, the network file is in '.R' format, but the data files we chose are in '.dat' format for testing.

The network file consists of various components.

First, we need to define the set of nodes and pipes terminated by ';'.

set nodes:= define the set of nodes here ;

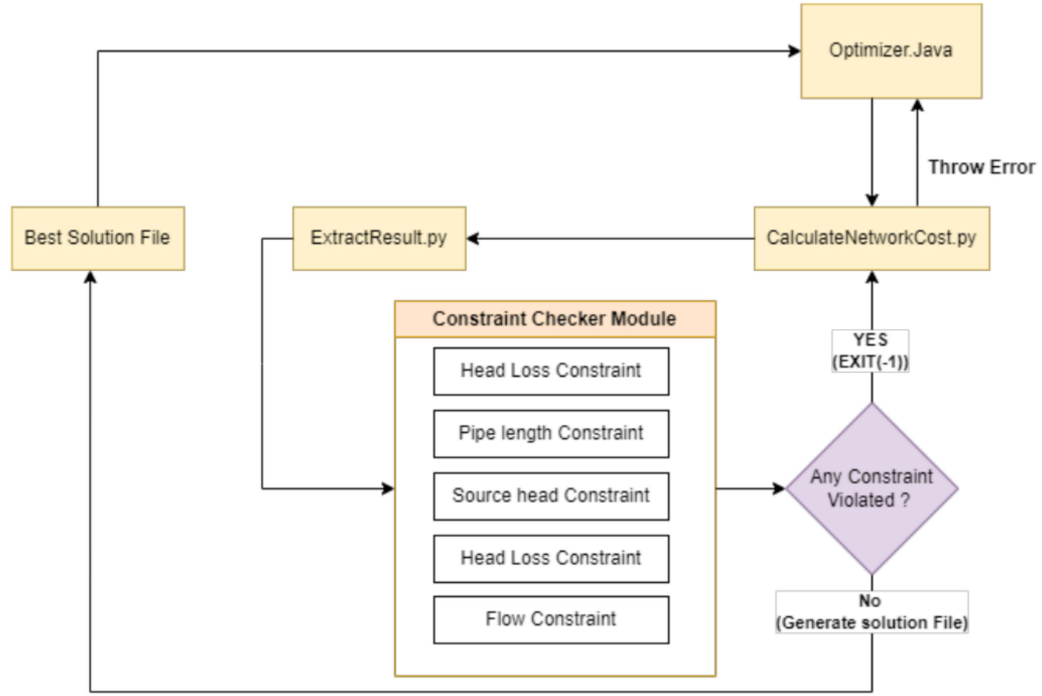


Figure 2.3: High-level overview of Jaltantra Constraint checker

set pipes:= define the set of pipes here ;

After defining the sets, we need to determine the necessary parameters that will be served in the model file. The parameters we define are L as the length of each arc (the link between the nodes), E as the Elevation of each node, P as the pressure at each node, D as the demand of each node, d as the diameter of each commercially available pipes, C as cost of each commercially available pipe, R as the roughness of each pipe. The format is given below.

```

param : arcs : L := node1 node2 length ;
param E := node elevation ;
param P := node pressure ;
param D := pipe demand ;
param d := pipe diameter ;
param C := pipe cost ;
param R := pipe roughness ;
param Source := source node ;

```

# Chapter 3

## Problem Formulation

### 3.1 Motivation

Jaltantra is a web system used to cost-optimize for building reliable and efficient water distribution systems. The input given to Jaltantra is a set of pipes, nodes, diameter, and cost of each commercially available pipe. Jaltantra optimizes this network while considering demand and pressure constraints. Various solvers are used to give optimal results through nonlinear problem-solving techniques. For the output, we consider multiple pipe segments per link. The output provides the length of selected pipe diameters in each link—the amount of flow between the nodes, velocity between the nodes, and overall network cost.

### 3.2 Problem Statement

In the Jaltantra loop, we use external nonlinear programming solvers Knitro, Alphaecp, and Baron to optimize the network. But these are commercially available solvers, and the license must be bought for commercial use. We have to choose one of the solvers for production use, so a thorough analysis of these solvers is required to test which solver gives the best result.

In the Jaltantra branch version, there is a feature of max allowed velocity in the network, but in the loop version, there is no constraint about velocity in the models. We aim to introduce a new velocity constraint to restrict the max allowed velocity in the network and optimize the cost of the network accordingly. The added constraint is nonlinear, which will help us analyze the solvers' performance and behavior.

# Chapter 4

## Methodology

The main objective of Jaltantra[6] is to optimize the cost of the network by selecting the appropriate pipe diameter for each link. Low pipe diameter results in lower cost but incurs higher head loss, therefore not satisfying the pressure constraints. Larger pipe diameters lower the head loss but increase the cost of the network. So, choosing the optimal pipe diameter is a necessary task. We use linear and nonlinear programming problem-solving techniques for optimization.

### 4.1 Model details

In Jaltantra, the objective functions and constraints are written in two model files, M1 and M2, which are in AMPL format. These two files are described below :

Some notations that will be useful are

- $l_{i,j,k} \rightarrow$  length of each pipe segment  $k$  between nodes  $i$  and  $j$
- $L_{i,j} \rightarrow$  length of link between node  $i$  and  $j$
- $A \rightarrow$  set of arcs
- $P \rightarrow$  set of Pipes
- $h_i \rightarrow$  head at node  $i$
- $E_i \rightarrow$  elevation of node  $i$
- $q_{i,j} \rightarrow$  flow from node  $i$  to node  $j$
- $C_k \rightarrow$  cost of commercially available pipe diameter

- $D_j \rightarrow$  Demand at node  $j$
- $R_k \rightarrow$  Roughness of pipe segment  $k$
- $d_k \rightarrow$  diameter of pipe segment  $k$
- $P_i \rightarrow$  pressure at node  $i$

#### 4.1.1 Model M1

**Objective Function :**

$$O(.) = \min \sum_{(i,j) \in A} \sum_{k \in P} l_{i,j,k} * C_k \quad (4.1)$$

**Flow Constraint :**

$$\sum_{(i,j) \in A} q_{i,j} = \sum_{(i,j) \in A} q_{j,i} + D_j \quad (4.2)$$

**Node/Head Constraint :**

$$h_i \geq E_i + P_i \quad (4.3)$$

$$h_i - h_j = \sum_{k \in P} \frac{10.68 * (q_{i,j} * |q_{i,j}|^{0.852}) * l_{i,j,k}}{R_k^{1.852} * d_k^{4.87}}, \text{ where } (i,j) \in A, k \in P \quad (4.4)$$

$$h_s = E_s \text{ where } s \in \text{Source} \quad (4.5)$$

**Pipe Length Constraint :**

$$\sum_{k \in P} l_{i,j,k} = L_{i,j} \text{ where } (i,j) \in A \quad (4.6)$$

#### 4.1.2 Model M2

In model m1, flow, i.e.,  $q_{i,j}$  can be both negative and positive. Positive flow indicates flow from node  $i$  to node  $j$ . Negative flow indicates the opposite direction of the flow, i.e., flow from node  $j$  to node  $i$ .

In model m2 we separate out positive and negative flows as  $q1$  and  $q2$ . So there is no negative flow in the model, i.e.,  $q1$  and  $q2$  are always positive. So, we have to add constraints that define these flows.

**Objective Function :**

$$O(.) = \min \sum_{(i,j) \in A} \sum_{k \in P} l_{i,j,k} * C_k \quad (4.7)$$

**Flow Constraint :**

$$\sum_{(i,j) \in A} (q1_{i,j} - q2_{i,j}) = \sum_{(i,j) \in A} (q1_{i,j} - q2_{i,j}) + D_j \quad (4.8)$$

$$q1_{i,j} \geq 0 \quad (4.9)$$

$$q2_{i,j} \geq 0 \quad (4.10)$$

$$q1_{i,j} * q2_{i,j} \leq q\_Max * q\_min \quad (4.11)$$

where,

- $q\_Max$  is max allowed flow
- $q\_min$  is min flow roughly tending to zero

**Node/Head Constraint :**

$$h_i \geq E_i + P_i \quad (4.12)$$

$$h_i - h_j = \sum_{k \in P} \frac{10.68 * (q1_{i,j}^{1.852} - q2_{i,j}^{1.852}) * l_{i,j,k}}{R_k^{1.852} * d_k^{4.87}}, \text{ where } (i, j) \in A, k \in P \quad (4.13)$$

$$h_s = E_s \text{ where } s \in Source \quad (4.14)$$

**Pipe Length Constraint :**

$$\sum_{k \in P} l_{i,j,k} = L_{i,j} \text{ where } (i, j) \in A \quad (4.15)$$

**Pipe Length Constraint :**

$$\sum_{k \in P} l_{i,j,k} = L_{i,j} \text{ where } (i, j) \in A \quad (4.16)$$

## 4.2 Introducing velocity constraints

The Jaltantra branch [6] had the feature of restricting the velocity between nodes by  $v_{max}$  provided by the design engineer. But in the loop version, a user can not limit the flow velocity between nodes. A new constraint must be added to ensure it does not violate the other constraints and gives optimal results. To understand this, we must first know how flow is defined between any two nodes. We know that

$$q_{i,j} = v_{i,j} * A_{i,j} \quad (4.17)$$

where,

- $q_{i,j} \rightarrow$  flow between nodes  $i$  and  $j$
- $v_{i,j} \rightarrow$  velocity between nodes  $i$  and  $j$
- $A_{i,j} \rightarrow$  Area of the pipe segment between nodes  $i$  and  $j$

Now, the Area of the pipe can be defined as,

$$A_{i,j} = \frac{\pi}{4} * d_{i,j}^2 \quad (4.18)$$

where,

- $d_{i,j} \rightarrow$  diameter of the pipe segment between nodes  $i$  and  $j$

But we don't have  $d_{i,j}$  directly, as we have multiple pipe segments per link. So, we consider the average value of these pipe segments for the diameter. Therefore, the pipe diameter will

$$d_{i,j} = \frac{\sum_{k \in P} d_k * l_{i,j,k}}{L_{i,j}} \quad (4.19)$$

Now, to restrict the velocity between any two nodes, we have to bind the flow between any two nodes by flow generated by max velocity between these nodes.

$$q_{i,j} \leq A_{i,j} * v_{max_{i,j}} \quad (4.20)$$

where,  $v_{max_{i,j}} \rightarrow$  max allowed velocity between nodes  $i$  and  $j$ .

- **M1 model**

The final equations will be added in M1 model

$$q_{i,j} \leq \frac{\pi}{4} * \left( \frac{\sum_{k \in P} d_k * l_{i,j,k}}{L_{i,j}} \right)^2 * v\_max_{i,j} \quad (4.21)$$

Also we need to add constraint for negative flows

$$q_{i,j} \geq -\left( \frac{\pi}{4} * \left( \frac{\sum_{k \in P} d_k * l_{i,j,k}}{L_{i,j}} \right)^2 * v\_max_{i,j} \right) \quad (4.22)$$

These constraints have to be added to the M1 model. The above constraints are nonlinear because the diameter is in quadratic form, adding more complexity to the model. So solving these problems is hard. We also need to add constraints in model M2. These are as follows :

- **M2 Model**

In model M2 we have to add constraints for  $q1_{i,j}$  and  $q2_{i,j}$

$$q1_{i,j} \leq \frac{\pi}{4} * \left( \frac{\sum_{k \in P} d_k * l_{i,j,k}}{L_{i,j}} \right)^2 * v\_max_{i,j} \quad (4.23)$$

$$q2_{i,j} \leq \frac{\pi}{4} * \left( \frac{\sum_{k \in P} d_k * l_{i,j,k}}{L_{i,j}} \right)^2 * v\_max_{i,j} \quad (4.24)$$

After adding these constraints, validation of these constraints is necessary. Also, we need to observe the performance of various solvers before and after adding the constraints. Also, we need to thoroughly compare the results to get the best solver to be chosen.

Detailed experimentation about these constraints is discussed in the next chapter. A thorough analysis of the results of these solvers is done in the results chapter.

# Chapter 5

## Experimentation

In this chapter, we will discuss the setups for the experimentation that has to be performed. First, we will discuss the system specifications on which the experiments have been performed. Then, we will discuss the data sets used in the experiments. After that, we will have an overview of AMPL and GAMS setups for experimentation, and then we will discuss the test cases on which the experiments will be performed.

### 5.1 System Specifications

#### 5.1.1 Hardware Requirements

- **Processor :** Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz
- **RAM :** 8GB
- **Storage Type :** SSD

#### 5.1.2 Software Requirements

Here we will explain only software needed for the experimentation that are performed.

- **OS :** Ubuntu (linux)
- **python 3.10.10**
- **AMPL :** Version 20211222 (Linux x86\_64 (gcc 7.5.0))

- **GAMS** : version 43.2

- **Available network files**

To evaluate solvers, we employed a set of eleven data files drawn from a diverse mix of synthetic and real-world network datasets. These files are listed below:

| <b>Data File</b> | <b>Name</b>         | <b>Size</b>           |
|------------------|---------------------|-----------------------|
| <b>d1</b>        | <i>Two Loop</i>     | 6(node), 8(links)     |
| <b>d2</b>        | <i>Cycle Hanoi</i>  | 31(nodes) ,34(links)  |
| <b>d3</b>        | <i>Double Hanoi</i> | 62(nodes), 67(links)  |
| <b>d4</b>        | <i>Triple Hanoi</i> | 92(nodes), 100(links) |
| <b>d5</b>        | <i>Taichung</i>     | 19(nodes), 31(links)  |
| <b>d6</b>        | <i>HG_SP_1_4</i>    | 78(node), 100(links)  |
| <b>d7</b>        | <i>HG_SP_2_3</i>    | 78, 100               |
| <b>d8</b>        | <i>HG_SP_3_4</i>    | 83, 99                |
| <b>d9</b>        | <i>HG_SP_4_2</i>    | 143, 199              |
| <b>d10</b>       | <i>HG_SP_5_5</i>    | 155, 200              |
| <b>d11</b>       | <i>HG_SP_6_3</i>    | 166, 198              |

Table 5.1: Dataset information

- **Available models**

In the Jaltantra model, a "Model" refers to a representation of the nonlinear program that is structured in a manner compatible with the AMPL interface. This model includes an objective function, the purpose of which is to minimize the costs associated with the water distribution network. Additionally, it comprises a series of constraints designed to ensure compliance with specific requirements, such as maintaining the necessary head, flow, and pressure at various nodes within the network. We have two model files namely

- **m1\_basic.R**
- **m2\_basic2\_v2.R**

- **Available solvers**

A solver is software that can exist as a standalone computer program or a programming library. Its primary function addresses mathematical problems

like linear and nonlinear programming problems. It takes the problem description, which should be in a standardized format, and then proceeds to identify the solution. We have three solvers namely

- KNITRO (AMPL based)
- BARON (AMPL based)
- ALPHAIECP (GAMS based)

So we have  $3(\text{solvers}) * 2(\text{models}) * 11(\text{data files}) = 66$  combinations that need to be tested for each iteration of varying parameters. We will try each combination with and without adding constraints for 1min and 5min, respectively.

## 5.2 Architecture for running solvers with models

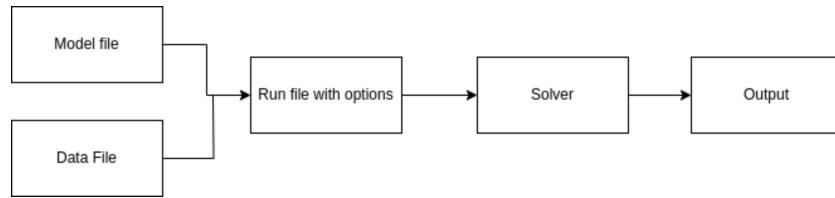


Figure 5.1: Solver methodology

Figure 5.1 shows a general architecture for running solvers with a given model and data file. We give the model and data file as input in the run file. We also specify the solver and other running options in the run file, then this run file is used to run with the given solver, and output is produced. But we must do this individually for each data and model file solver combination. Therefore, we need to automate the process to get the output.

## 5.3 AMPL experimentation setup

AMPL [5], short for "A Mathematical Programming Language," is a versatile language designed for algebraic modeling. It enables modeling a wide range of programming problems, including linear and nonlinear ones, using imperative and declar-

ative programming styles. This language is compatible with various solvers that support AMPL modeling.

The solvers that we are using from available solvers in AMPL are KNITRO [9] and BARON [8]. We will use these solvers with models m1 and m2, with added constraints and without adding constraints. The experiments to be run are for 1min and 5min. Figure 5.2 shows artitechture for performing the AMPL experiments.

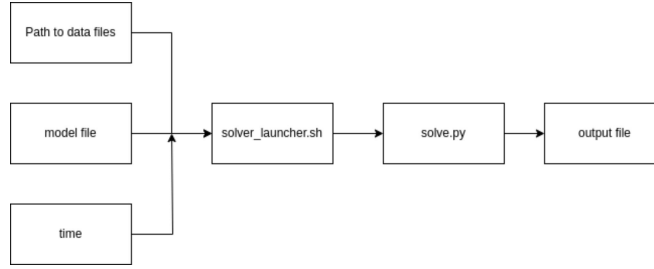


Figure 5.2: Architechture for AMPL solvers

The description of the above programs is as follows:

- **solver\_launcher.sh** : This bash script takes the folder path of the data files, model file path, and time. Then, it launches solve.py for each model-data file combination.
- **solve.py** : This Python script takes data and model file as command line args and creates a temporary run file as per AMPL format [5], which has the necessary command to run this model and data file with specified solver along with options. Then, we invoke the AMPL process with the run file, and the output is written to the file whose name is the combination of model, data, and time.
- **output** : The output file is in AMPL output format. The parameters we take for consideration are objective cost and vmax, which are written to [Excel sheet](#), which are compared further.

## 5.4 GAMS experimentation setup

Modeling in GAMS is different from AMPL. In AMPL, we can use model and data files independently, but in GAMS, we have to create a '.gms' file, which consists

of data, modeling, and options for solving nonlinear programming in the same file. So we must create this '.gms' file for each model and data combination separately per GAMS[4] docs. Then, this '.gms' is given to the GAMS solver, and output is produced in '.lst' format. Again, we must convert this '.lst file' for AMPL-like output.

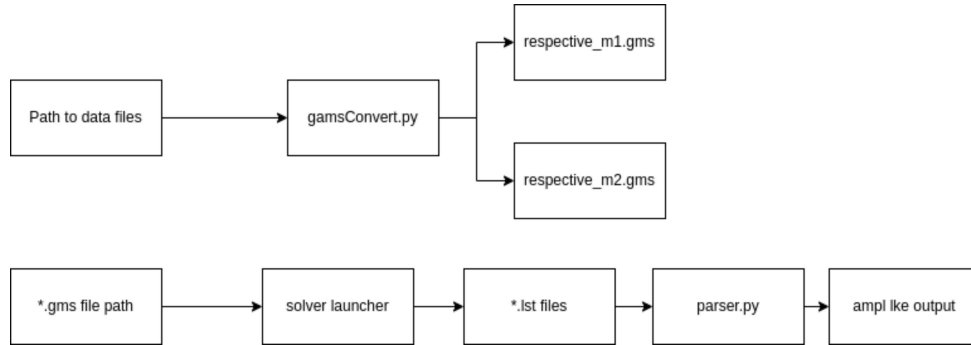


Figure 5.3: Architecture for GAMS solvers

The architecture diagram for experimenting is shown in fig 5.3. The solver we will use with GAMS is ALPHAIECP [3]. We will run it for 1 min and 5 min each model with and without constraint and analyze the results. The description of the programs is as follows:

- **gamsConvert.py** : This Python script takes the data file as input. It first creates two files for m1 and m2 models. Then the data file with respective '.gms' file is passed to **parse\_dat\_to\_gms** function, which parses the data file with the help of regex and writes to the respective gams file according to GAMS format [4]. Then, each file, m1 and m2 are written with the constraints and solving options.
- **solver\_launcher.sh** : We give the path of the folder that contains all these '.gms' files to 'solver\_launcher'; it then iterates over all the files in the folder and launches GAMS solver for each file. Then the output is in the form '.lst,' and we have to pass it to AMPL-like format. So we use 'parser.py' to convert gams output to AMPL output.

## Chapter 6

# Improving Velocity Constraint

### 6.1 Issues with average velocity constraint

We used the average area of the pipe segments in the link (i,j) to estimate the flow through the link, with the intention of bounding the velocity. However, using the average area does not accurately represent the actual flow dynamics.

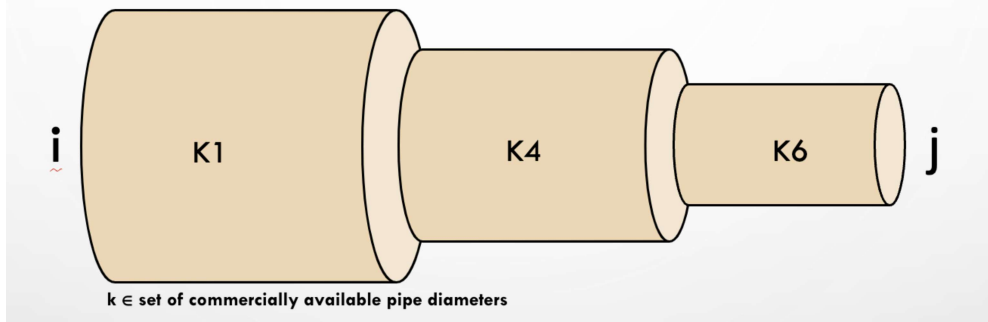


Figure 6.1: Link between nodes i & j consist of multiple pipes of varying diameters

Jaltantra employs a multiple-pipe-segment per link model as shown in fig [6.1](#), which significantly impacts the network's design and analysis. This model allows each link between nodes in the network to consist of several pipe segments, each potentially having a different diameter. As a result, the solution obtained after solving the network does not assume a single, uniform pipe diameter for any given link. Instead, it can incorporate multiple pipes of varying diameters selected from a predefined set of available diameters for that link.

### 6.1.1 Velocity and flow dynamics in pipe segments

In the context of Jaltantra’s multiple-pipe-segment per link model, it is important to understand the relationship between flow rate, cross-sectional area, and velocity within the network. Despite the complexity introduced by varying pipe diameters, the flow rate remains constant across any given link (i,j). This consistency ensures that the volume of water flowing through the link does not change, regardless of the individual pipe segments’ diameters.

However, the cross-sectional area of the pipe segments within a link can vary. This variation is a key factor in the network’s hydraulic behavior. Specifically, the fluid velocity is inversely related to the cross-sectional area of the pipe segments:

- **Higher Cross-Sectional Area:** When a segment has a larger cross-sectional area, the fluid velocity in that segment will be lower. This occurs because the same volume of water is distributed over a larger area, reducing the speed at which it travels.
- **Lower Cross-Sectional Area:** Conversely, when a segment has a smaller cross-sectional area, the fluid velocity will be higher. In this case, the same volume of water is confined to a smaller space, increasing its speed.

### 6.1.2 Limitations of using average velocity for bounding

In Jaltantra, relying on average velocity to bound the actual velocity can be misleading and insufficient. The average velocity does not accurately reflect the variations across different segments within a link, potentially masking critical issues.

For instance, consider a link where the maximum allowable velocity,  $v_{max}$ , is 5 m/s. If the velocities in various segments of the link are:

- Velocity in segment  $k1 = 1$  m/s
- Velocity in segment  $k4 = 5$  m/s
- Velocity in segment  $k6 = 9$  m/s

The average velocity for these segments is  $(1 + 5 + 9)/3 = 5$  m/s. While this average is within the allowable limit, the velocity in segment  $k6$  significantly exceeds  $v_{max}$  reaching 9 m/s.

This example illustrates the inadequacy of using average velocity as the sole metric for bounding the velocity in the network. It highlights the necessity for

a more detailed analysis that considers the velocity in each individual segment, ensuring that none exceed the specified limits.

## 6.2 Bounding velocity in selected pipe segments

### 6.2.1 Challenges of bounding velocity in all pipe segments

In the context of Jaltantra's multiple-pipe-segment per link model, ensuring that velocity constraints are met across all pipe segments can be challenging. Consider the following scenario:

- Total pipe segments: 10
- Selected pipe segments in the link: 3 (k1, k4, k6)
- Non-selected pipe segments: 7 (k2, k3, k5, k7, k8, k9)

If we impose velocity bounds on all pipe segments, some of the non-selected pipe segments may violate the velocity constraint. For instance, while segments k1, k4, and k6 adhere to the velocity limits, segments from the non-selected group (k2, k3, k5, k7, k8, k9) might exceed the allowable velocity. Such violations can lead to an infeasible solution, as the network fails to uniformly meet the required velocity constraints.

To effectively manage this, we need a clear representation for selected pipe segments in the link. Introducing a binary variable can help in this regard.

### 6.2.2 Binary variable to represent the selected pipes in the link

To represent the pipes that are selected in the link  $(i, j)$ , we introduce a binary variable  $z_{i,j,k}$ . This variable is defined as follows:

$$z_{i,j,k} = \begin{cases} 1 & \text{when } k^{\text{th}} \text{ pipe in } (i, j)^{\text{th}} \text{ link is used, i.e. } l[i, j, k] \text{ is non-zero} \\ 0 & \text{else} \end{cases} \quad (6.1)$$

Using this binary variable, we can effectively track and manage which specific pipe segments are utilized within each link of the network.

- When  $z_{i,j,k} = 1$ , it indicates that the  $k^{\text{th}}$  pipe is part of the link  $(i, j)$ .
- When  $z_{i,j,k} = 0$ , it means that the pipe segment is not used.

### 6.2.3 Determining binary variable constraints

We need to form the constraints for determining the value of the binary variable  $z_{i,j,k}$ . One constraint should determine the value of  $z_{i,j,k}$  to be 1 when the pipe segment is actually used in the link (i,j). The value of . Other constraint would determine the value of the binary variable to be zero when the other pipe segments are not used. The constraints are as follows:

**Determining  $z_{i,j,k}$  to be 1 when  $l_{i,j,k}$  is non zero**

$$z_{i,j,k} \geq \frac{l_{i,j,k}}{L_{i,j}} \quad (6.2)$$

Equation 6.14 ensures that  $z_{i,j,k}$  is 1 , when  $l_{i,j,k}$  is non-zero.

**Determining  $z_{i,j,k}$  to be zero when  $l_{i,j,k}$  is zero**

Here we have two equations that which can be discussed further.

$$(L_{i,j} * \text{Min\_frac\_link\_length}) * z_{i,j,k} \leq l_{i,j,k} \quad (6.3)$$

The parameter `Min_frac_link_length` signifies the fraction of the total link length that dictates the number of pipes allowed in the link (i,j). This parameter needs to be fine-tuned on an ad-hoc basis to suit specific network requirements.

Assuming `Min_frac_link_length` is set to 0.1 and the total link length  $L[i,j]=1000$ , Substituting the given values in eq 6.3, we get:

$$(1000 \cdot 0.1 \cdot z[i, j, k]) \leq l[i, j, k] \quad (6.4)$$

This simplifies to:

$$100 \cdot z[i, j, k] \leq l[i, j, k] \quad (6.5)$$

This means that for a pipe segment to be selected in the network ( $z[i,j,k]=1$ ), its length  $l[i,j,k]$  must be at least 100 meters. This constraint ensures that each selected pipe segment meets a minimum length requirement. However, the assumption that a selected pipe segment must be greater than 100 meters may not always hold true.

To ensure that  $z[i,j,k]$  is zero when  $l[i,j,k]$  is zero, we modify the constraint as follows:

$$z_{i,j,k} \leq L_{i,j} \cdot l_{i,j,k} \quad (6.6)$$

This condition is sufficient to determine  $z[i,j,k]$  to be zero when  $l[i,j,k]$  is zero. By this constraint, if  $l[i,j,k]$  is zero, then the right-hand side of the inequality becomes zero, forcing  $z[i,j,k]$  to be zero as well. This ensures that the binary variable  $z[i,j,k]$  correctly reflects the usage of the pipe segment  $l[i,j,k]$  in the link  $(i,j)$ .

Eq 6.6 does not look dimensionally correct as  $z[i,j,k]$  is unitless and  $l[i,j,k] * L[i,j]$  is  $m^2$ . To ensure that the constraint for  $z[i,j,k]$  is dimensionally correct, we modify it as follows:

$$z_{i,j,k} * d_k^2 \leq L_{i,j} * l_{i,j,k} * 1e6 \quad (6.7)$$

Here,  $d[k]$  is the diameter of the pipe segment. This adjustment ensures that both sides of the equation are in  $m^2$ , making the equation dimensionally consistent. This constraint effectively sets  $z[i,j,k]$  to zero when,  $l[i,j,k]$  is zero.

#### 6.2.4 Improving the velocity constraints

We have previously discussed how average velocity does not accurately represent the actual picture of bounding the velocity within a link. To address this, we introduced a binary variable to represent the pipes that are actually selected in the link. Using this binary variable, we can form more precise velocity constraints.

The average velocity constraint was originally expressed as:

$$q_{i,j} \leq \frac{\pi}{4} \left( \frac{\sum_{k \in P} d_k \cdot l_{i,j,k}}{L_{i,j}} \right)^2 \cdot v_{max_{i,j}} \quad (6.8)$$

The modified constraints, which incorporate the binary variable, are as follows:

$$\frac{q_{i,j}}{\frac{\pi}{4} d_k^2} \leq v_{max_{i,j}} + M_{big}(1 - z_{i,j,k}) \quad (6.9)$$

$$\frac{q_{i,j}}{\frac{\pi}{4} d_k^2} \geq -(v_{max_{i,j}} + M_{big}(1 - z_{i,j,k})) \quad (6.10)$$

Here,  $M_{big}$  is a large constant used to bound the velocity when  $z[i,j,k]$  is zero. When  $z[i,j,k]$  is 1, indicating that the  $k$ -th pipe in the link  $(i,j)$  has been selected, the velocity will be bounded by  $v_{max}$ . Conversely, when the pipe segment is not selected ( $z[i,j,k]=0$ ), the velocity is effectively unbounded within the limits imposed by  $M_{big}$ . This approach ensures accurate velocity constraints for selected pipe segments while accommodating unselected segments without violating network integrity.

These constraints can be narrowed down to

$$\frac{abs(q_{i,j})}{\frac{\pi}{4}d_k^2} \leq v_{max_{i,j}} + M_{big}(1 - z_{i,j,k}) \quad (6.11)$$

### Issues with M\_big constraint

The use of M\_big in the constraints requires careful fine-tuning and is often an ad-hoc solution. To address these issues, we propose a modified constraint that incorporates the maximum flow q\_M in the network, which is determined by the demand at the source. The original constraints can be refined as follows:

$$abs(q_{i,j}) \leq v_{max} \cdot \frac{\pi}{4} \cdot d_k^2 \cdot z_{i,j,k} + q_M \cdot (1 - z_{i,j,k}) \quad (6.12)$$

Here, q\_M represents the maximum flow in the network, a known quantity based on the demand at the source. This modified constraint ensures that when z[i,j,k] is 1, the flow, q[i,j] is bounded by v\_max and the diameter d[k] of the selected pipe segment. When z[i,j,k] is 0, the flow is bounded by q\_M, effectively eliminating the need for the ad-hoc tuning of M\_big.

## 6.2.5 Constraints for considerations

The constraints for considerations for are as follows :

- The velocity constraint with binary variable

$$abs(q_{i,j}) \leq v_{max} \cdot \frac{\pi}{4} \cdot d_k^2 \cdot z_{i,j,k} + q_M \cdot (1 - z_{i,j,k}) \quad (6.13)$$

- z[i,j,k] constraints

$$z_{i,j,k} \geq \frac{l_{i,j,k}}{L_{i,j}} \quad (6.14)$$

$$z_{i,j,k} * d_k^2 \leq L_{i,j} * l_{i,j,k} * 1e6 \quad (6.15)$$

## 6.2.6 Results from testing

We have tested these constraints on 11 data files and the results were as follows :

From table [6.1](#), we can observe the following points:

| Data File          | Name         | Size                  | No velocity constraint (Baron) |          | No velocity constraint (Knitro) |          | With velocity constraint (Baron) |          | With velocity constraint (Knitro) |          |
|--------------------|--------------|-----------------------|--------------------------------|----------|---------------------------------|----------|----------------------------------|----------|-----------------------------------|----------|
|                    |              |                       | 5min                           |          | 5min                            |          | 5min                             |          | 5min                              |          |
| d1                 | Two Loop     | 6(node), 8(links)     |                                | 4.04E+05 |                                 | 4.04E+05 |                                  | 4.09E+05 |                                   | 4.17E+05 |
| d2                 | Cycle Hanoi  | 31(nodes), 34(links)  |                                | 6.06E+06 |                                 | 6.06E+06 | optimum found at node -3         |          |                                   | 6.06E+06 |
| d3                 | Double Hanoi | 62(nodes), 67(links)  |                                | 1.21E+07 |                                 | 1.21E+07 |                                  | 1.24E+07 |                                   | 1.21E+07 |
| d4                 | Triple Hanoi | 92(nodes), 100(links) |                                | 1.84E+07 |                                 | 1.84E+07 | optimum found at node -3         |          |                                   | 1.84E+07 |
| d5                 | Taichung     | 19(nodes), 31(links)  | optimum found at node -3       |          | 8.82E+06                        |          | optimum found at node -3         |          |                                   | 8.84E+06 |
| d6                 | HG_SP_1,4    | 78(node), 100(links)  |                                | 3.55E+06 |                                 | 3.56E+06 |                                  | 3.74E+06 | locally infeasible                |          |
| d7                 | HG_SP_2,3    | 78, 100               | optimum found at node -3       |          | 3.40E+06                        |          | optimum found at node -3         |          |                                   | 3.40E+06 |
| d8                 | HG_SP_3,4    | 78, 100               |                                | 3.55E+06 |                                 | 3.56E+06 |                                  | 3.74E+06 | infeasible point                  |          |
| d9                 | HG_SP_4,2    | 83,99                 |                                | 6.92E+06 |                                 | 6.92E+06 |                                  | 6.94E+06 | infeasible point                  |          |
| d10                | HG_SP_5,3    | 155, 200              |                                | 9.37E+06 |                                 | 9.38E+06 | optimum found at node -3         |          | locally infeasible                |          |
| d11                | HG_SP_6,3    | 166, 198              |                                | 1.13E+07 |                                 | 1.08E+07 | optimum found at node -3         |          | locally infeasible                |          |
| feasible solutions |              |                       | 9                              |          | 11                              |          | 5                                |          | 6                                 |          |

Table 6.1: Results of velocity constraints testing

- **No Velocity Constraints:** Without applying the velocity constraints, Baron produces feasible solutions for 9 out of the 11 data files. Specifically, Baron finds infeasible solutions for the network files d5 and d7. In contrast, Knitro delivers feasible solutions for all the data files when no constraints are applied.
- **With Velocity Constraints:** Upon applying the velocity constraints, the performance differs. Baron is able to provide feasible results for only 5 out of the 11 data files. Knitro, on the other hand, manages to find feasible solutions for 6 out of the 11 data files under the same conditions.

### 6.2.7 Complexity analysis

To understand why we did not obtain feasible solutions when velocity constraints were applied, we conducted further analysis on the increase in the number of variables and constraints. The complexity analysis is as follows:

From the table [6.2](#), we can observe that the number of variables almost doubles, and the total number of constraints increases significantly. This substantial increase in problem size negatively impacts the performance of the solver. Specifically, the introduction of binary variables leads to an exponential increase in complexity. This surge in complexity is a primary factor contributing to the inability to find feasible solutions in some cases when velocity constraints are applied.

## 6.3 Optimizing binary variable constraints

In the previous section, we observed that the complexity of the problem increases exponentially when binary variables are introduced, and constraints are applied to all arcs. This significant increase in complexity often leads to infeasible solutions and decreased solver performance.

|          | Without Velocity constraint |             | With Velocity All arcs |             |
|----------|-----------------------------|-------------|------------------------|-------------|
| Datafile | Variables                   | Constraints | Variables              | Constraints |
| d1       | 125                         | 22          | 233                    | 263         |
| d2       | 262                         | 93          | 465                    | 499         |
| d3       | 519                         | 185         | 919                    | 1097        |
| d4       | 775                         | 276         | 1371                   | 1808        |
| d5       | 453                         | 82          | 856                    | 888         |
| d6       | 1362                        | 263         | 2562                   | 2994        |
| d7       | 1366                        | 267         | 2566                   | 3003        |
| d8       | 1362                        | 263         | 2562                   | 2994        |
| d9       | 1333                        | 245         | 2517                   | 2847        |
| d10      | 2733                        | 531         | 5133                   | 5084        |
| d11      | 2708                        | 534         | 5084                   | 6587        |

Table 6.2: Effect of applying velocity constraints on number of variables and total number of constraints

We propose optimizing performance by selectively applying the velocity constraints to address this. Specifically, we will use these constraints only to the arcs that violated the maximum velocity condition in the previous solution. This targeted approach reduces the number of variables and constraints, thereby mitigating the exponential increase in complexity.

Additionally, we will use the solution obtained without velocity constraints as the initial starting point for the next iteration. This strategy leverages previously obtained feasible solutions to guide the solver, potentially improving convergence and overall performance.

This iterative approach balances the need for adhering to velocity constraints with the practical necessity of maintaining solver efficiency and feasibility.

Fig 6.2 demonstrates the constraints are applied on violated arcs only. We build an iterative system using these constraints as a base. The flow chart for the iterative system is shown in fig 6.3

### 6.3.1 The iterative system

Bounding velocity in violated arcs may further violate the constraint in some other arcs. So, we need to bound the velocity in arcs iteratively until we get a feasible

```

73 #bounding velocity on violated arcs only
74 subject to cond8{(i,j) in v_arcs, k in pipes}:
75     (abs(q[i,j])/1000) <= (vmax[i,j])*((3.14/4)*(d[k]/1000)^2)*z[i,j,k] + q_M*(1 - z[i,j,k]);
76
77 #Ensuring z[i,j,k] to be zero when kth pipe segment is not selected
78 subject to cond12{(i,j) in v_arcs, k in pipes}:
79     z[i,j,k]*(d[k]^2) <= L[i,j] * l[i,j,k]*1e6;
80
81 #Ensuring z[i,j,k] to be 1 when kth pipe segment is selected
82 subject to con14{(i,j) in v_arcs, k in pipes}:
83     z[i,j,k] >= l[i,j,k] / L[i,j];

```

Figure 6.2: Constraints are applied on violated arcs only

solution. Therefore we have build an iterative system in which maximum velocity constraints are applied on violated arcs only and we feed the solution of previous iteration as a starting point for next iteration. The step by step details of iterative system is as follows :

- **STEP 1** : Initially we load the data file with empty set of violated arcs and initialize the *iter* variable to zero as a counter for iteration.
- **STEP 2** : Run the solver with data file with empty *v\_arcs* and model file
- **STEP 3** : Check if feasible solution obtained
  - If infeasible solution found, then return to user with message "No feasible solution found" meaning that we will not get feasible solution even after applying velocity constraints.
  - If feasible solution found, then we check the velocity in all arcs.
- **STEP 4** : Check if velocity is bounded in all arcs
  - If velocity in all is below the maximum allowable velocity in the arcs, means the velocity is bounded in all arcs. Then we return the solution to user with message "Feasible solution with bounded velocity".
  - Else we begin the next iteration by bounding velocity in violated arcs only.
- **STEP 5** : Begin the iteration and increment the *iter* variable.
- **STEP 6** : Check if *iter* is less than *MAX\_ITER*

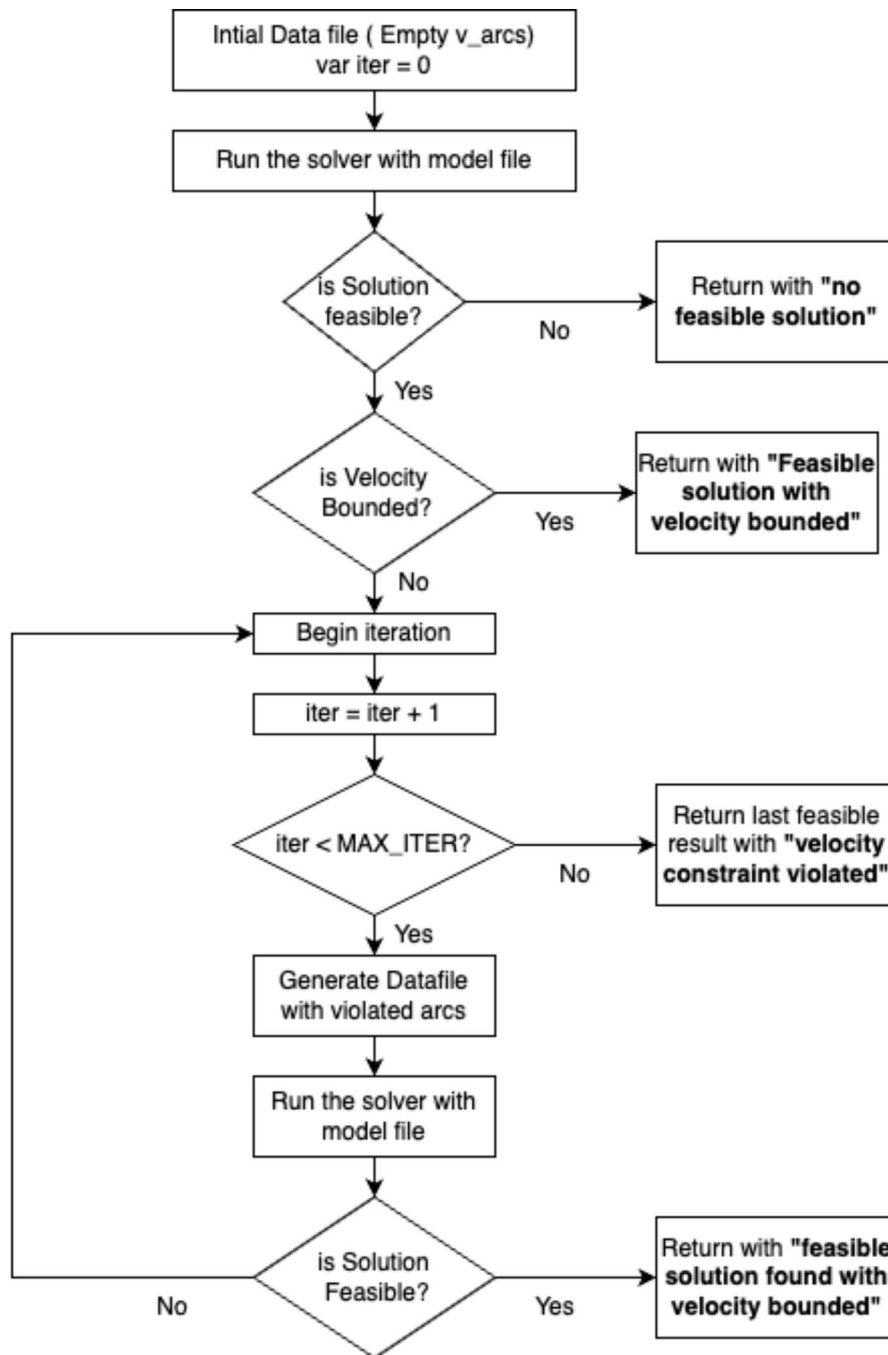


Figure 6.3: Flowchart for iterative solution

- If it exceeds the maximum allowable iterations, means that we did not get feasible solution by bounding the velocity. So we return the last feasible solution with a message "velocity constraint violated."

- Else generate new data file with set of arcs which exceeded the maximum allowable velocity in the network.
- **STEP 7** : Generate new data file
- **STEP 8** : Run the solver again with new data file and model
- **STEP 9** : Again check if solution is feasible after applying the constraint on violated arcs only
  - If the solution is feasible, then return to user with message "feasible solution found with velocity bounded"
  - Else, repeat step 5

### 6.3.2 Results of iterative system

We conducted experiments using the iterative system on 11 network files, varying the maximum allowable velocity. Initially, we recorded the maximum velocities in each network file. Then, we tested bounding the velocities at 20 m/s, 15 m/s, and 10 m/s. The results of these experiments are detailed below:

#### Velocities in each data file without velocity constraints

In the table [6.3](#) we can find that maximum velocity obtained from all data files is 19.60m/s without applying the velocity constraint. So we will start applying velocity constraints on all data files with initial  $v_{max}$  as 20m/s and observe the results.

#### Bounding velocity in each data file by $v_{max}$ as 20m/s

In this experiment we will set the  $v_{max}$  parameter to 20m/s and run the experiment of iterative system. The results are shown in table [6.4](#)

We can observe from table [6.4](#), we are able to bound the velocity in network files by 20m/s in the first iteration only.

#### Bounding velocity with $v_{max}$ 15m/s

In this experiment we will set the  $v_{max}$  parameter to 15m/s and run the experiment of iterative system. The results are shown in table [6.5](#).

| Datafile | min_velocity(m/s) | max velocity (m/s) |
|----------|-------------------|--------------------|
| d1       | 0.37              | 2.29               |
| d2       | -1.62             | 6.83               |
| d3       | -1.62             | 13.36              |
| d4       | -2.47             | 19.6               |
| d5       | -2.38             | 2.42               |
| d6       | -6.23             | 4.86               |
| d7       | -4.72             | 4.57               |
| d8       | -6.23             | 4.86               |
| d9       | -6.32             | 6.33               |
| d10      | -6.1              | 4.32               |
| d11      | -6.5              | 4.07               |

Table 6.3: Maximum and minimum velocities in each network file without velocity constraints

|          | KNITRO              |             |            |          |
|----------|---------------------|-------------|------------|----------|
| Datafile | Is Velocity bounded | Is Feasible | Iterations | Cost     |
| d1       | yes                 | yes         | 1          | 4.04E+05 |
| d2       | yes                 | yes         | 1          | 6.06E+06 |
| d3       | yes                 | yes         | 1          | 1.21E+07 |
| d4       | yes                 | yes         | 1          | 1.84E+07 |
| d5       | yes                 | yes         | 1          | 8.83E+06 |
| d6       | yes                 | yes         | 1          | 3.55E+06 |
| d7       | yes                 | yes         | 1          | 3.40E+06 |
| d8       | yes                 | yes         | 1          | 3.55E+06 |
| d9       | yes                 | yes         | 1          | 6.92E+06 |
| d10      | yes                 | yes         | 1          | 9.35E+06 |
| d11      | yes                 | yes         | 1          | 1.07E+07 |

Table 6.4: Bound velocity with vmax 20m/s

We can observe in table [6.5](#), we are able to bound the velocity all network files. In the first iteration of d4 when we did not apply constraints and run solvers, and got

|          | KNITRO              |             |            |          |
|----------|---------------------|-------------|------------|----------|
| Datafile | Is Velocity bounded | Is Feasible | Iterations | Cost     |
| d1       | yes                 | yes         | 1          | 4.04E+05 |
| d2       | yes                 | yes         | 1          | 6.06E+06 |
| d3       | yes                 | yes         | 1          | 1.21E+07 |
| d4       | no                  | yes         | 1          | 1.84E+07 |
| d4_iter1 | yes                 | yes         | 2          | 1.84E+07 |
| d5       | yes                 | yes         | 1          | 8.83E+06 |
| d6       | yes                 | yes         | 1          | 3.55E+06 |
| d7       | yes                 | yes         | 1          | 3.40E+06 |
| d8       | yes                 | yes         | 1          | 3.55E+06 |
| d9       | yes                 | yes         | 1          | 6.92E+06 |
| d10      | yes                 | yes         | 1          | 9.35E+06 |
| d11      | yes                 | yes         | 1          | 1.07E+07 |

Table 6.5: Bounding velocity with vmax 15m/s

infeasible solution, but the velocities in some arcs were exceeding the max allowable velocity. In the next iteration we are able to bound the velocity in network file d4 and got feasible solution.

### Bounding velocity with vmax 10m/s

In this experiment we will set the vmax parameter to 15m/s and run the experiment of iterative system. The results are shown in table 6.6

We can observe in the table 6.6 we are able to bound the velocity and get feasible solution in 1st iteration only for all the network files except d4 and d3. For d4, it needed 2 iterations to get the feasible solution and bound the velocity. In case of d3 we are not able to bound the velocity even after 5 iterations. Further analysis has to be done with data file d3, for why it gave infeasible solution.

### 6.3.3 Analysis of why d3 not able to bound the velocity

In the table 6.6 we have seen that in network file d3, velocity could not be bounded even after 5 iteration. In order to understand we did complete analysis of the network file and the calculations are shown in table 6.7 The Analysis is as follows:

- Max velocity in d3 without applying velocity constraints = 13.36 m/s

|          | KNITRO              |             |            |          |
|----------|---------------------|-------------|------------|----------|
| Datafile | Is Velocity bounded | Is Feasible | Iterations | Cost     |
| d1       | yes                 | yes         | 1          | 4.04E+05 |
| d2       | yes                 | yes         | 1          | 6.06E+06 |
| d3       | no                  | yes         | 1          | 1.21E+07 |
| d3_iter1 | no                  | yes         | 2          | 1.21E+07 |
| d3_iter2 | no                  | yes         | 3          | 1.21E+07 |
| d3_iter3 | no                  | yes         | 4          | 1.21E+07 |
| d3_iter4 | no                  | yes         | 5          | 1.21E+07 |
| d4       | no                  | yes         | 1          | 1.84E+07 |
| d4_iter1 | yes                 | yes         | 2          | 1.84E+07 |
| d5       | yes                 | yes         | 1          | 8.83E+06 |
| d6       | yes                 | yes         | 1          | 3.55E+06 |
| d7       | yes                 | yes         | 1          | 3.40E+06 |
| d8       | yes                 | yes         | 1          | 3.55E+06 |
| d9       | yes                 | yes         | 1          | 6.92E+06 |
| d10      | yes                 | yes         | 1          | 9.35E+06 |
| d11      | yes                 | yes         | 1          | 1.07E+07 |

Table 6.6: Bounding velocity with vmax 10m/s

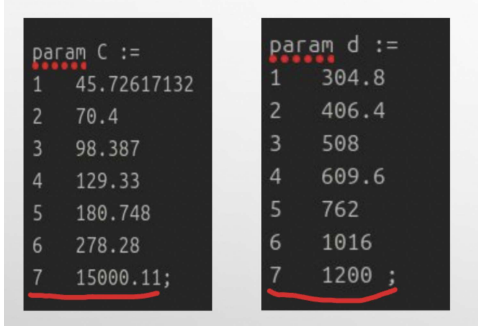
- We want to bound velocity by 10m/s
- The dataset we have, contains the max available diameter as 1016 mm
- No feasible solution due to insufficient available diameters despite velocity constraint
- A feasible solution requires at least one available diameter over 1174.57mm

| For d3 why not able to bound                                      |             | Formula               |
|-------------------------------------------------------------------|-------------|-----------------------|
| q[1,2] Fixed as 1 is source(ltr.)                                 | 10830       | Given in data file    |
| Unit conversion( $m^3/s$ )                                        | 10.83       | $q[1,2]/1000$         |
| Area( $m^2$ )                                                     | 1.083       | $q[1,2]=A*v_{max}$    |
| Minimum diameter required(mm)                                     | 1174.571341 | $A=(PI/4)*(d/1000)^2$ |
| All diameter available are less than this , hence unable to bound |             |                       |

Table 6.7: d3 Analysis to find minimum diameter required

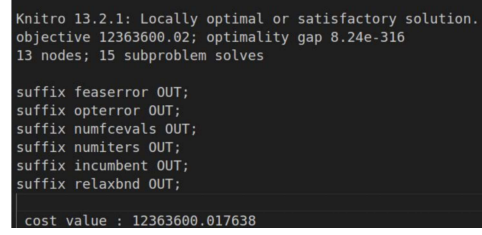
### Modifying network file to get required solution

To test our hypothesis we have included a new (ad-hoc) diameter 1200mm with an increased cost Rs. 1500.11/mm in datafile d3.



| param C :=    | param d := |
|---------------|------------|
| 1 45.72617132 | 1 304.8    |
| 2 70.4        | 2 406.4    |
| 3 98.387      | 3 508      |
| 4 129.33      | 4 609.6    |
| 5 180.748     | 5 762      |
| 6 278.28      | 6 1016     |
| 7 15000.11;   | 7 1200 ;   |

Figure 6.4: Modified d3



```
Knitro 13.2.1: Locally optimal or satisfactory solution.
objective 12363600.02; optimality gap 8.24e-316
13 nodes; 15 subproblem solves

suffix feaseerror OUT;
suffix opterror OUT;
suffix numfcevals OUT;
suffix numiters OUT;
suffix incumbent OUT;
suffix relaxbnd OUT;

cost value : 12363600.017638
```

Figure 6.5: Knitro Result

Fig 6.4 represents modified d3 data file. We have run the Knitro solver with model file and modified d3 data file. Result of analysis is shown in fig 6.5. We can observe that we get after modifying the data file i.e adding the necessary pipe diameters in the set.

Hence we can say that velocity bounding depends on the set of available pipe diameters. And to bound the velocity in the network with low velocity we might need to include sufficient diameters as per the previous calculations.

### 6.3.4 Issues with Iterative System

The iterative system, although effective in bounding velocities in network pipes, introduces some significant overheads. Specifically:

- **Solver Running Time:** The solver running time can become unnecessarily high if the velocity is not bounded between the links. This iterative approach may require multiple solver runs, each time adjusting the constraints based on the previous solution, which can be computationally expensive and time-consuming.

### Proposed Solutions

To mitigate the issues associated with the iterative system, we propose the following solutions:

- **Apply Constraints on All Arcs:** Instead of iteratively applying constraints only on violated arcs, we can apply the velocity constraints on all arcs from the beginning. This approach may reduce the number of iterations required to find a feasible solution, potentially improving overall solver performance.
- **Remove Dependency on Binary Variables:** To further optimize performance, we suggest removing the dependency on binary variables. Instead, we can use continuous variables to represent the constraints. This modification can significantly reduce the complexity of the problem, leading to faster and more efficient solutions.

## 6.4 Converting binary variable to continuous variable

So far we have seen various constraints for bounding the velocity between links by maximum allowable velocity  $v_{max}$ . We have discussed the results in details and the drawbacks of these constraints. Initially we discussed applying velocity constraints with binary variable on all arcs settings. Then we found that after applying these constraints the time complexity increases exponentially and got infeasible solution for many of the files.

To optimize the results we did some changes like applying the constraints on violated arcs only and feeding the previous solution as a starting point for next iteration. In this way we build the iterative system. But with iterative system, the solver running time increases unnecessarily if we are not able to bound the velocity.

To overcome all these drawbacks we can think of making the constraints by eliminating the dependency of binary variable and introducing continuous variable. We will also apply these constraints on all arcs only. The derivation for constraints is as follows :

We know that,

$$q_{i,j} = A_k * v \quad (6.16)$$

To bound the flow with maximum velocity

$$q_{i,j} \leq A_k * v_{max} \quad (6.17)$$

Multiplying  $l_{i,j,k}$  on both sides

$$l_{i,j,k} * (q_{i,j}) \leq l_{i,j,k} * (A_k * v_{max}) \quad (6.18)$$

Rearranging the terms

$$l_{i,j,k} * (q_{i,j} - A_k * v_{max}) \leq 0 \quad (6.19)$$

As flow can be negative also

$$l_{i,j,k} * (abs(q_{i,j}) - \frac{PI}{4} * d_k^2 * v_{max}) \leq 0 \quad (6.20)$$

Eq 6.20 represents the constraint for bounding the velocity. Here  $l_{i,j,k}$  is length of the kth pipe segment in the link (i,j).  $l_{i,j,k}$  is the continuous variable, which determines the behaviour of this constraint.

- When  $l_{i,j,k}$  is zero, the LHS of eq 6.20 becomes zero and satisfies the constraint.
- When  $l_{i,j,k}$  is non-zero, the constraint  $(q_{i,j} \leq A_k * v_{max})$  will be enforced.

In this way, we can obtain the desired results by eliminating the need for binary variable and using the continuous variable  $l[i,j,k]$ .

#### 6.4.1 Results after applying continuous variable constraint

To validate the newly formed continuous variable constraints with binary variable, we performed analysis on 15 data files. Out of 15 data files, 11 are the old data files d1-d11, we modified d3 and d4 to d3\_verify.dat and d4\_verify.dat by adding necessary diameters required to satisfy the velocity constraints. We also added two new data files which are Indian data networks, d12 and d13. Data file d12 is a large network consisting of 200 nodes and d13 is mid scale network consisting of 70 nodes. The results after experimentation is shown in table 6.8

From the table 6.8, we can observe that

- We are able to get the feasible solution for all the data files when maximum allowable velocity is 20m/s.
- We are also able to get the feasible solution for all data files when maximum allowable velocity is 10m/s except old d3 and d4, reason being lack of necessary pipe diameters in the set. When we modified d3 and d4 by adding the necessary pipe diameters in the set we are able to bound the velocity as well as get the feasible solution.

|                  | Knitro      |          |             |          |
|------------------|-------------|----------|-------------|----------|
|                  | V_max=20m/s |          | V_max=10m/s |          |
| DataFile         | is_feasible | cost     | is_feasible | cost     |
| d1               | yes         | 4.04E+05 | yes         | 4.04E+05 |
| d2               | yes         | 6.06E+06 | yes         | 6.06E+06 |
| d3               | yes         | 1.21E+07 | no          | 1.72E+07 |
| <b>d3_verify</b> | yes         | 1.21E+07 | yes         | 1.23E+07 |
| d4               | yes         | 1.85E+07 | no          | 2.55E+07 |
| <b>d4_verify</b> | yes         | 1.84E+07 | yes         | 1.87E+07 |
| d5               | yes         | 8.83E+06 | yes         | 8.82E+06 |
| d6               | yes         | 3.55E+06 | yes         | 3.55E+06 |
| d7               | yes         | 3.40E+06 | yes         | 3.40E+06 |
| d8               | yes         | 3.55E+06 | yes         | 3.55E+06 |
| d9               | yes         | 6.92E+06 | yes         | 6.92E+06 |
| d10              | yes         | 9.41E+06 | yes         | 9.43E+06 |
| d11              | yes         | 1.15E+07 | yes         | 1.07E+07 |
| d12              | yes         | 1.46E+07 | yes         | 1.46E+07 |

Table 6.8: Continuous variable constraint result

- In the table [6.9](#) we can observe we are able to get feasible solution in data files d12 and d13 with maximum allowable velocities as 5m/s and 3m/s.

| (Indian WDN) | V_max=5m/s  |          | V_max=3m/s  |          |
|--------------|-------------|----------|-------------|----------|
| DataFile     | is_feasible | cost     | is_feasible | cost     |
| d12          | yes         | 1.46E+07 | yes         | 1.46E+07 |
| d13          | yes         | 2.25E+06 | yes         | 2.25E+06 |

Table 6.9: Continuous variable velocity constraint results on Indian WDN

In this we have verified the velocity constraint with max velocity in the next section we will try to derive constraints for minimum velocity in the network.

## 6.5 Minimum velocity constraints

In the previous section we successfully implemented and tested the maximum velocity constraint ie. to bound the velocity in each arcs by maximum allowable velocity given by the user. In this section we will try to derive the constraints for the minimum velocity constraint. Our aim is, velocity in arc should be greater than or equal to minimum velocity in the network  $vmin$ . The derivations is same as derivation of max velocity constraint.

We want the flow in link should be greater than the minimum flow obtained by  $vmin$ .

$$q_{i,j} \geq A_k * vmin \quad (6.21)$$

$$q_{i,j} \geq \frac{PI}{4} * d_k^2 * vmin \quad (6.22)$$

Multiplying  $l_{i,j,k}$  on both the sides

$$l_{i,j,k} * (q_{i,j}) \geq l_{i,j,k} * \left( \frac{PI}{4} * d_k^2 * vmin \right) \quad (6.23)$$

Rearranging the terms

$$l_{i,j,k} * \left( q_{i,j} - \frac{PI}{4} * d_k^2 * vmin \right) \geq 0 \quad (6.24)$$

Eq [6.24](#) represent the constraint which enforces the velocity in the link has to be greater than or equal to minimum velocity.

### 6.5.1 Testing minimum velocity constraint

First we run the solvers with models without any velocity constraints and notes the minimum velocity obtained in the network. Then we perform the experimenters by by including the maximum and minimum velocity constraints in the model. We varied the  $vmin$  as 0m/s, 0.01m/s, 0.1m/s and 0.6m/s and noted the results.

Table [6.10](#) presents the minimum velocities obtained across all data files when velocity constraints are not included in the model. It is evident that the majority of these minimum velocities are of the order of  $1e-4$ , indicating extremely small values. To further our analysis, we conducted additional experiments by incorporating both minimum and maximum velocity constraints into the model.

| min-velocity (m/s) |                         |
|--------------------|-------------------------|
| DataFile           | without vmin constraint |
| d1                 | 3.80E-01                |
| d2                 | 2.10E-01                |
| d3                 | 2.02E-01                |
| <b>d3_verify</b>   | 2.02E-01                |
| d4                 | 1.51E-02                |
| <b>d4_verify</b>   | 1.51E-02                |
| d5                 | 1.21E-01                |
| d6                 | 5.62E-04                |
| d7                 | 1.38E-03                |
| d8                 | 5.62E-04                |
| d9                 | 7.86E-04                |
| d10                | 8.44E-04                |
| d11                | 4.24E-04                |
| d12                | 4.23E-03                |
| d13                | 2.31E-02                |
| <b>vmin all</b>    | 4.24E-04                |

Table 6.10: Minimum velocity in all the data files

|                            | V_max=20m/s   |          |               |          |              |          |              |          |
|----------------------------|---------------|----------|---------------|----------|--------------|----------|--------------|----------|
|                            | V_min=0.00m/s |          | V_min=0.01m/s |          | V_min=0.1m/s |          | V_min=0.6m/s |          |
| DataFile                   | is_feasible   | cost     | is_feasible   | cost     | is_feasible  | cost     | is_feasible  | cost     |
| d1                         | yes           | 4.04E+05 | yes           | 4.04E+05 | yes          | 4.17E+05 | yes          | 4.26E+05 |
| d2                         | yes           | 6.06E+06 | yes           | 6.18E+06 | yes          | 6.09E+06 | yes          | 6.42E+06 |
| d3                         | yes           | 1.21E+07 | yes           | 1.27E+07 | yes          | 1.24E+07 | no           |          |
| <b>d3_verify</b>           | yes           | 1.21E+07 | yes           | 1.25E+07 | yes          | 1.29E+07 | yes          | 1.28E+07 |
| d4                         | yes           | 1.85E+07 | yes           | 2.02E+07 | no           |          | no           |          |
| <b>d4_verify</b>           | yes           | 1.84E+07 | yes           | 1.94E+07 | yes          | 1.90E+07 | yes          | 3.12E+07 |
| d5                         | yes           | 8.83E+06 | yes           | 9.18E+06 | yes          | 9.38E+06 | yes          | 9.51E+06 |
| d6                         | yes           | 3.55E+06 | no            |          | no           |          | no           |          |
| d7                         | yes           | 3.40E+06 | yes           | 3.40E+06 | no           |          | no           |          |
| d8                         | yes           | 3.55E+06 | no            |          | no           |          | no           |          |
| d9                         | yes           | 6.92E+06 | no            |          | no           |          | no           |          |
| d10                        | yes           | 9.41E+06 | no            |          | no           |          | no           |          |
| d11                        | yes           | 1.15E+07 | no            |          | no           |          | no           |          |
| d12                        | yes           | 1.46E+07 | yes           | 1.57E+07 | no           |          | no           |          |
| d13                        | yes           | 2.25E+06 | yes           | 2.25E+06 | no           |          | no           |          |
| <b>#feasible solutions</b> | 15            |          | 10            |          | 6            |          | 5            |          |

Table 6.11: Result of applying minimum velocity constraints

Table 6.11 shows the results for various network files after applying minimum velocity constraints. We experimented with different settings for the minimum velocity ( $v_{\min}$ ), and observed the following points:

- **$v_{\min} = 0.0$  m/s:** All velocities in each pipe segment of the link must be greater than 0 m/s. This setting resulted in feasible solutions for all data files.
- **$v_{\min} = 0.01$  m/s:** Feasible solutions were obtained for 10 network files, while 5 network files failed to produce feasible solutions.
- **$v_{\min} = 0.1$  m/s:** Only 6 network files were able to produce feasible solutions, with the rest failing to satisfy the constraints.
- **$v_{\min} = 0.6$  m/s:** Feasible solutions were obtained for 5 network files, while 10 data files produced infeasible solutions.

These observations indicate that further research is needed to redesign the minimum velocity constraints to achieve more consistent results.

# Chapter 7

## Feature Enhancements

### 7.1 Integrating new velocity constraint in Jaltantra

Until now, we have verified the velocity constraint and analyzed the performance of various solvers after adding the constraint. In this chapter, we will see how I have integrated the new constraints into the Jaltantra system.

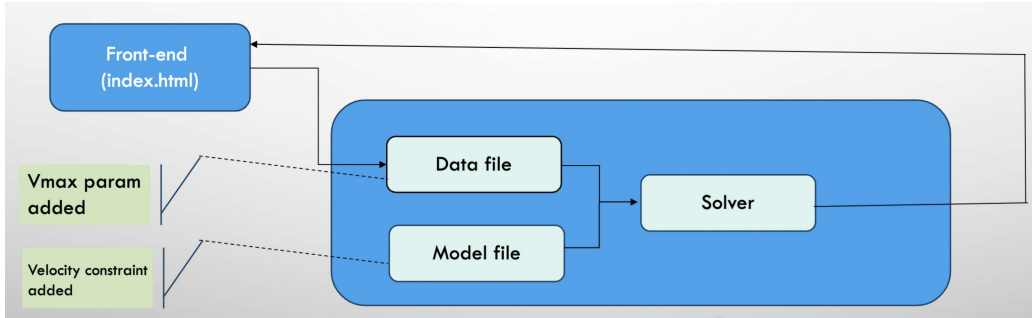


Figure 7.1: Simplified Jaltantra Architecture

Figure [7.1](#) represents simplified Jaltantra architecture. We need to do some changes in the code in order to integrate the velocity constraint in the system. These changes are discussed below.

#### 7.1.1 Changes needed to do in code

- **index.html** : I have added a new field called **max water speed**, which takes the max velocity that has to be incorporated for the whole network. This field is also passed with the JSON request that is passed to the **Optimizer servlet**.

- **OptimizerServlet :** In the constructor we define **vmax** as the value passed by the user. We take the default value as 10000 if the user provides no value.
- **Optimizer.java :** In this program we create the network file. We have to define the **vmax** as a parameter in the data network file. So we add **v\_max** for every pipe arc defined by the user in the **createNetworkFile()** method. We also add the **vmax** while creating network files for GAMS. We write our necessary constraints in the method **createGamsModel**.
- **Model Files :** We have added the necessary constraints in the model files in the path `Jaltantra-code-and-Scripts/Files/Models/m1_basic.R` and `m2_basic2_v2.R`.

## 7.2 Notification improvement

In the current deployment of the Jaltantra-system, we encountered an issue with the notification system when solvers were unable to produce a feasible solution. Specifically, the notification prompt would stop with 0:00:05 seconds remaining, causing confusion and uncertainty about the status of the computation. This issue needs to be addressed to ensure clear and accurate notifications are provided, especially in cases of infeasible solutions. Figure [7.2](#) represent the screenshot of Jaltantra system when feasible solution is not found.

### 7.2.1 Key improvements

Figure [7.3](#) shows necessary improvements done in the notification in case of infeasible solution given by the solver.

- **Accurate Notifications for Infeasible Solutions:** The system now clearly notifies users when a solver cannot find a feasible solution. This eliminates the confusion caused by the previous countdown timer stopping prematurely.
- **Extended Runtime in the Background:** To increase the chances of finding a solution, the system will automatically rerun the same file for an extended period in the background. This helps ensure that if a solution is possible within a reasonable time frame, it will be found and reported.
- **User Tips and Guidance:** Users will receive suggestions on what actions to take next if an infeasible solution is found. This includes reviewing input

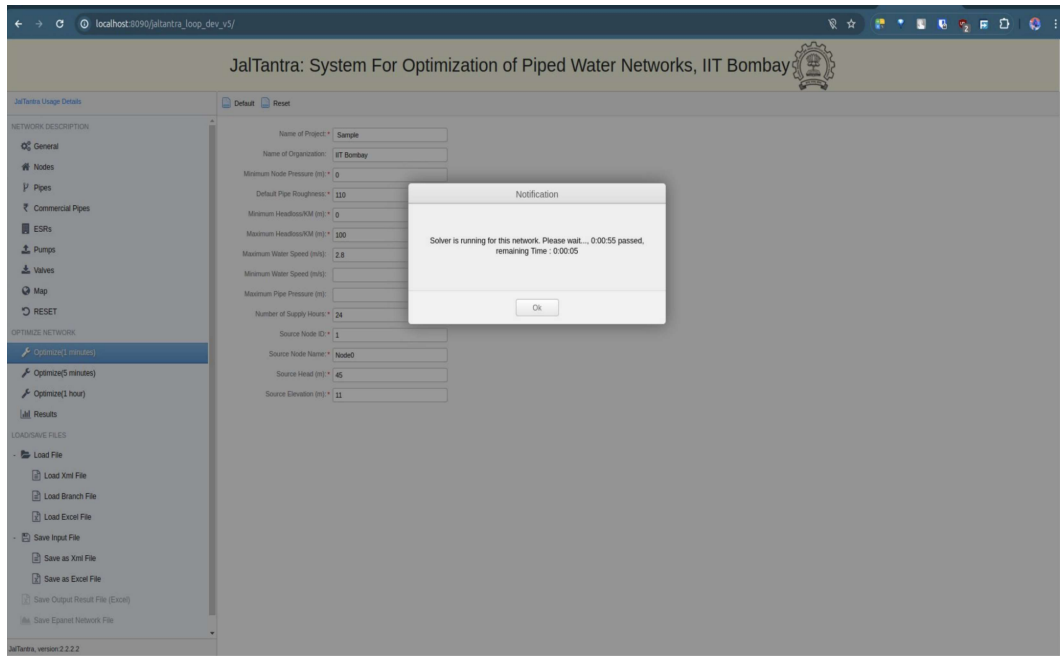


Figure 7.2: Current Jaltantra System with issue in notification

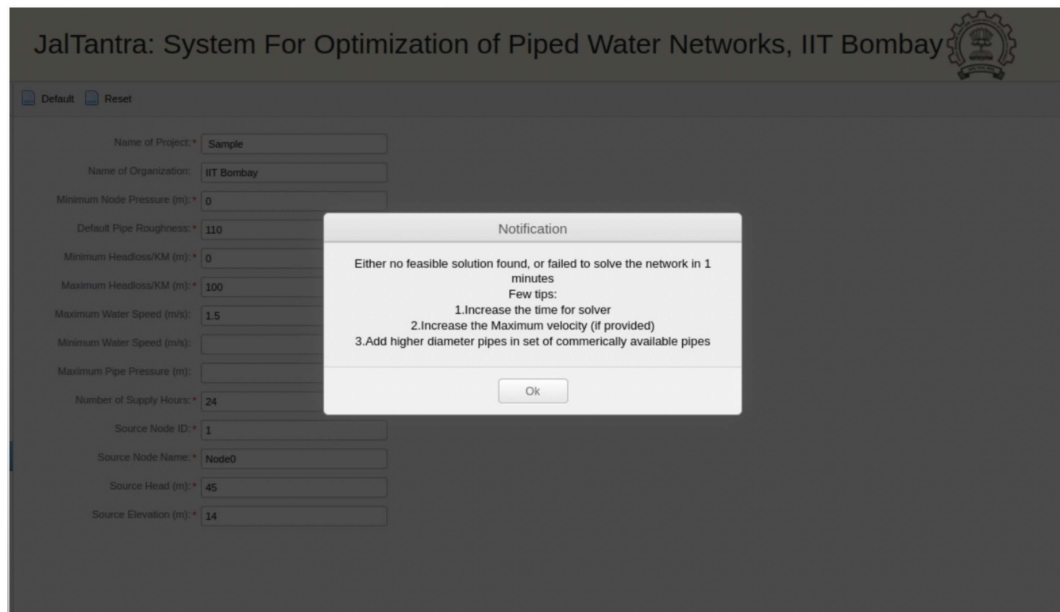


Figure 7.3: Improved notification in case of infeasible solution

parameters and constraints or trying different solver settings. Some of probable reasons for infeasible solution is shown to user, and some suggestions are

given as follows:

- One of the probable reason for infeasible solution could be lack of sufficient time for the solver to solve the network. We suggest user to increase solver time in the next attempt.
  - Also not proving sufficient velocity could also be the reason so that user got infeasible solution. It can we also one of the try to increase the max velocity and check the solution.
  - As per the previous calculations shown to find minimum diameter requirement for lower velocity bounding, user need to add sufficient diameters in the set of commercially available pipe diameters.
- **Seamless User Experience:** When the user clicks next time to check the results, the system will present the findings if a solution has been found during the extended runtime. This ensures users are informed without having to wait excessively.

## 7.3 Automation of initial setup of Jaltantra

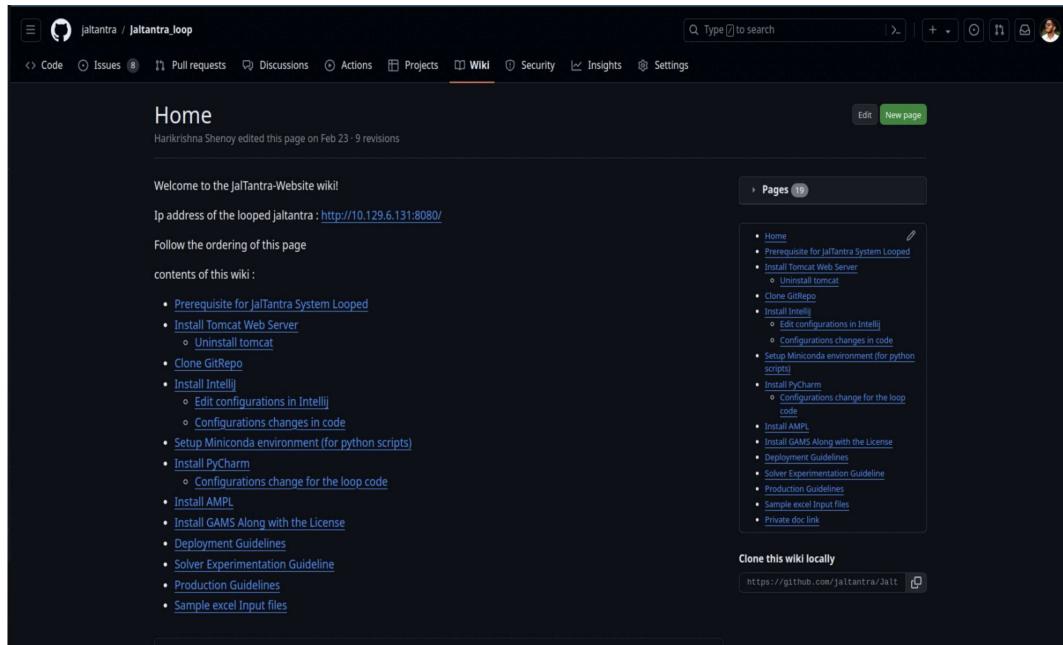


Figure 7.4: Installation Guidelines for Jalatantra System

Figure [7.4](#) shows guidelines and documentation for installing Jaltantra system of the local system of developer. It has various challenges discussed below.

### 7.3.1 Challenges

- **Tedious Installation Process:** Setting up Jaltantra manually is time-consuming and complex.
- **Prone to Human Errors:** The manual steps involved increase the risk of errors.
- **IDE Requirements:** The setup requires specific Integrated Development Environments (IDEs) like IntelliJ and PyCharm.
- **Tomcat Installation and Configuration:** Setting up the Tomcat server and configuring daemon processes adds to the complexity.

### 7.3.2 Automation Steps (`initial_setup.sh`)

Setting up Jaltantra manually can be a complex and error-prone process. To streamline this, we have developed an automated setup script that simplifies the installation and configuration of all necessary components. This ensures a consistent and efficient setup, allowing developers to focus on writing code rather than dealing with system configurations.

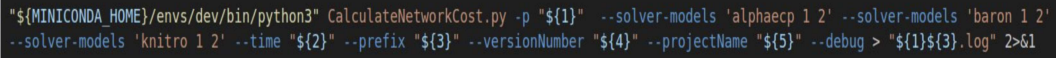
The `initial_setup.sh` script automates the following steps:

- **Tmux Installation :** Updates the system packages and installs tmux for terminal multiplexing.
- **Miniconda Setup :** Downloads and installs Miniconda, sets up the dev environment, and installs necessary Python packages.
- **MySQL Setup :** Installs MySQL server, creates a user and database, and configures the application properties.
- **Configuration of application.properties :** Modifies the `application-dev.properties` file to include the MySQL credentials and solver directory path.
- **Copy AMPL :** Securely copies AMPL from the Jaltantra server and sets the appropriate permissions.

The automation script provides a one-click solution for setting up Jaltantra on new devices, significantly reducing the manual effort and potential for errors. Developers can now focus on writing code without worrying about the system and configuration setup. Additionally, all hard-coded dependencies have been removed to enhance flexibility and portability.

## 7.4 Removing AlphaECP solver from Jaltantra

In the first part of the thesis, we have proved that Knitro is give the overall best result and decided to purchase the license for Knitro solver. We do not have license for AlphaECP solver, so we decided to remove the AlphaECP solver based on the analysis. But removing AlphaECP solver was a tedious task, as there were lot of things hardcoded and interdependencies made removing AlphaECP a challenging task.



```
"${MINICONDA_HOME}/envs/dev/bin/python3" CalculateNetworkCost.py -p "${1}" --solver-models 'alphaecp 1 2' --solver-models 'baron 1 2'
--solver-models 'knitro 1 2' --time "${2}" --prefix "${3}" --versionNumber "${4}" --projectName "${5}" --debug > "${1}${3}.log" 2>&1
```

Figure 7.5: CalculateNetworkCost\_JaltantraLauncher.sh (Script to invoke solver and run analysis)

Figure [7.5](#) represents the script CalculateNetworkCost\_JaltantraLauncher.sh which invokes all the solver to solve the network. We have to remove dependency of AlphaECP from the code written in the file CalculateNetworkCost.py.

### 7.4.1 Challenges

- **Fixed Order of Invoking Solvers:** The order in which solvers were invoked was hardcoded. This rigidity prevented us from easily removing or reordering solvers as needed.
- **Hardcoded Dependencies:** The system had numerous hardcoded dependencies that tightly coupled various components, making it challenging to isolate and remove specific solvers without affecting the overall functionality.

### 7.4.2 Solution

- **Refactoring Code:** We refactored the code to decouple the solver-specific logic. This involved identifying and abstracting the sections of code that were

specific to AlphaEcp, making it easier to remove or replace these sections without disrupting the entire system.

- **Flexible Solver Management:** We implemented a more flexible approach to managing solvers, allowing for dynamic inclusion or exclusion of solvers based on configuration files. This eliminated the need for hardcoded solver orders and dependencies.
- **Modular Design:** The system was redesigned to support a modular approach where solvers could be added or removed as independent modules. This design ensures that future changes to the solver lineup can be made with minimal impact on the rest of the codebase.

By addressing these issues, we improved the maintainability and flexibility of the Jaltantra system, allowing it to adapt more easily to future changes and requirements.

# Chapter 8

## Conclusion and Future Work

### 8.1 Conclusion

The initial work started with thoroughly understanding the currently deployed Jaltantra System. The code base is large and complex to understand, so we must understand each component properly. Each architectural component has been discussed in detail. We also know that three nonlinear programming solvers, namely Knitro, Aphaecp, and Baron, are integrated into the current Jaltantra system. We use these solvers on community licenses. For the production use of Jaltantra, detailed analysis of these solvers is a must regarding cost optimization and giving feasible solutions.

The new velocity constraint being nonlinear makes the model more complex. So, it helped to analyze the performance and behavior of various solvers. We have built programs that automate testing all the solver model combinations. It was found that Knitro M1 was giving promising results when we added the constraint. Other solvers like Baron did not produce answers with more data files, added conditions, and the M2 model. Alphaecp delivered a feasible solution with added constraints with both models, but its cost value was considerably higher than other solvers. It is safe to say that Knitro M1 is producing the best results till now. But still, it was giving more objective cost value when the dataset size was large, so further analysis has to be done with a knitro solver about this behavior.

Finally, we have integrated the new velocity constraint into the currently deployed Jaltantra system and Decided to remove Alphec solver from the system. Hence, this thesis helped to optimize the network cost by introducing the velocity constraint. Also we redesigned the code base to be more robust and resilient. We automated the process of initial installation of Jaltantra which took considerable

time if done manually, helping future developers to write code rather than setting up the system. This work has the potential to enhance the efficiency and cost-effectiveness of water network design significantly, ultimately benefiting engineers across the nation.

## 8.2 Future Work

The currently deployed system uses Java Servlets programs on the Tomcat 9 server. Both the Java Servlets and Tomcat are out-dating with the time. Tomcat 10 [2] does not support Javax libraries, so we need to migrate to new Jakarta libraries [1]. My partner Harikrishna has migrated the system to Spring Boot so these dependencies are resolved now.

The code structure in the project is also monolith, and even a single change in the project requires a lot of effort, so refactoring of the code must also be done to bring modularity. This will help the future developers to work on the project efficiently.

More research has to be done with the minimum velocity constraints. There are issues with the Google Map API needed to be fixed in future. More constraints can be added to minimize the headloss in the system.

# Bibliography

- [1] Apache Tomcat® - Apache Tomcat 10 Software Downloads.
- [2] Apache Tomcat® - Migration Guide - Tomcat 10.0.x.
- [3] GAMS Development Corporation. Gams alphacp solver documentation, 2023.
- [4] GAMS Development Corporation. Gams documentation, 2023.
- [5] Robert Fourer, David M. Gay, and Brian W. Kernighan. *AMPL: A Modeling Language for Mathematical Programming*. Duxbury Press, 2002.
- [6] Nikhil Hooda and Om Damani. A system for optimal design of pressure constrained branched piped water networks. *Procedia Engineering*, 186:349–356, 2017.
- [7] Nikhil Hooda and Om Damani. Jaltantra: A system for the design and optimization of rural piped water networks. *INFORMS Journal on Applied Analytics*, 49(6):447–458, 2019.
- [8] AMPL Optimization LLC. Baron documentation, 2022.
- [9] AMPL Optimization LLC. Knitro documentation, 2022.